

gormz — Complete Reference & Practical Guide

Type-safe, immutable ORM for Go — built on top of GORM

Version: 0.1.0 · **License:** MIT · **Author:** Sanad Team **Repository:**
<https://github.com/light-tech-dev/gormz>

Table of Contents

Part 1 — Foundations

1. [Introduction](#)
2. [Installation & Setup](#)
3. [Understanding GORM First](#)
4. [Your First gormz Query](#)
5. [Core Concepts In Depth](#)
6. [Defining Models](#)

Part 2 — Daily Usage 7. [Reading Data \(Queries\)](#) 8. [Writing Data \(Mutations\)](#) 9. [Advanced Lookups — Complete Guide](#) 10. [Q Builder — Complex Conditions](#) 11. [Pagination](#) 12. [Relations & Preloading](#) 13. [Aggregations](#)

Part 3 — Professional Usage 14. [Context & Cancellation](#) 15. [Multiple Databases](#) 16. [Model Registry](#) 17. [Transactions In Depth](#) 18. [Bulk Operations](#)

Part 4 — Advanced 19. [CTEs](#) 20. [Window Functions](#) 21. [Subqueries](#) 22. [Unions](#) 23. [Locking — Pessimistic & Optimistic](#) 24. [Retry with Backoff](#) 25. [Batch Processing](#)

Part 5 — Production 26. [Error Handling](#) 27. [Security](#) 28. [Performance Tuning](#) 29. [Migration from GORM](#)

Part 6 — Complete Project 30. [Building a Blog API — Full Walkthrough](#)

Appendices

- [A. Full API Reference](#)
 - [B. FAQ](#)
-

Part 1 — Foundations

1. Introduction

What is gormz?

gormz is a **type-safe, immutable ORM layer** for Go, built on top of [GORM](#). It's not a replacement — it's a **cleaner interface** for the same engine.

The core idea: instead of writing verbose GORM queries, you write concise, chainable, type-safe queries that read like natural language.

The problem gormz solves

With raw GORM:

```
var users []User
query := db.Model(&User{}).Where("active = ?", true)
if search != "" {
    query = query.Where("name LIKE ?", "%"+search+"%")
}
```

```

}
if minAge > 0 {
    query = query.Where("age >= ?", minAge)
}
if sortBy != "" {
    query = query.Order(sortBy + " DESC")
}
err := query.Limit(20).Offset(0).Find(&users).Error
if err != nil { /* handle */ }

```

With gormz:

```

q := gormz.New[User]().Filter("active", true)
if search != "" {
    q = q.Filter("name__contains", search)
}
if minAge > 0 {
    q = q.Filter("age__gte", minAge)
}
if sortBy != "" {
    q = q.OrderBy("-" + sortBy)
}
users, err := q.Limit(20).All()

```

Better in every way:

- Type-safe (compiler catches mistakes)
- No string concatenation
- Shorter and clearer
- Immutable (thread-safe)
- SQL injection safe

Key features at a glance

Feature	Description
Type-safe	QuerySet[T] with generics
Advanced lookups	__gt, __in, __contains, ...

Immutable	Every method returns a new copy
Thread-safe	Safe concurrent use
Context-first	Full <code>context.Context</code> support
Multi-DB	Multiple connections via <code>Instance</code>
Pagination	Built-in
Registry	Avoid import cycles
Typed errors	<code>NotFoundError</code> , <code>ValidationError</code> , ...
SQL injection safe	Field validation
Dangerous ops guards	No accidental <code>DELETE</code> without conditions
Advanced	CTEs, Windows, Unions, Subqueries
Locking	Pessimistic + Optimistic
Retry	Exponential/Linear/Constant backoff
Bulk ops	Insert, Upsert, Update, Delete
Batch processing	Parallel workers, streaming

Requirements

- **Go:** 1.22 or later (generics required)
 - **GORM:** v1.25 or later
 - **Database:** Any GORM-supported (SQLite, PostgreSQL, MySQL, SQL Server, ...)
-

2. Installation & Setup

Step 1 — Install gormz

```
go get github.com/light-tech-dev/gormz
```

Step 2 — Install a database driver

Choose your database:

SQLite (easiest for development):

```
go get gorm.io/driver/sqlite
```

PostgreSQL:

```
go get gorm.io/driver/postgres
```

MySQL:

```
go get gorm.io/driver/mysql
```

Step 3 — Create your project

```
mkdir myapp  
cd myapp  
go mod init github.com/yourname/myapp
```

Step 4 — Write the connection code

main.go:

```
package main  
  
import (  
    "log"  
  
    "github.com/light-tech-dev/gormz"  
    "gorm.io/driver/sqlite"  
    "gorm.io/gorm"
```

```

)

type User struct {
    ID      uint    `gorm:"primaryKey"`
    Name    string  `gorm:"size:255;not null"`
    Email   string  `gorm:"uniqueIndex;size:255"`
    Age     int
}

func (User) TableName() string { return "users" }

func main() {
    // Step 1: Open GORM connection
    db, err := gorm.Open(sqlite.Open("myapp.db"), &gorm.C
    if err != nil {
        log.Fatal("Failed to connect:", err)
    }

    // Step 2: Bind to gormz
    gormz.SetDB(db)

    // Step 3: Migrate
    if err := gormz.Migrate[User](); err != nil {
        log.Fatal("Migration failed:", err)
    }

    // Step 4: Use it
    user := &User{Name: "Ali", Email: "ali@test.com", Age
    if err := gormz.New[User]().Create(user); err != nil
        log.Fatal(err)
    }

    log.Printf("Created user ID=%d\n", user.ID)
}

```

Run it:

```
go run main.go
```

Output:

```
2026/09/28 15:30:00 Created user ID=1
```

Step 5 — Configure the connection pool

For production, tune the pool:

```
cfg := gormz.DefaultConfig()
cfg.MaxOpenConns = 50
cfg.MaxIdleConns = 10
cfg.ConnMaxLifetime = 2 * time.Hour
cfg.ConnMaxIdleTime = 30 * time.Minute
cfg.LogLevel = gormz.LogError
cfg.SlowQuery = 200 * time.Millisecond

if err := gormz.Configure(cfg); err != nil {
    log.Fatal(err)
}
```

Presets

Preset	Use for	MaxOpenConns	LogLevel
DefaultConfig()	Production	25	Silent
DevelopmentConfig()	Development	10	Info
TestingConfig()	Tests	5	Silent

Verify connection

```
if err := gormz.Ping(); err != nil {
    log.Fatal("DB unreachable:", err)
}
```

```
if !gormz.IsReady() {  
    log.Fatal("DB not initialized")  
}
```

Clean shutdown

```
defer gormz.Close()
```

Using PostgreSQL

```
import (  
    "gorm.io/driver/postgres"  
    "gorm.io/gorm"  
)  
  
dsn := "host=localhost user=postgres password=secret dbname=  
db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})  
gormz.SetDB(db)
```

Using MySQL

```
import "gorm.io/driver/mysql"  
  
dsn := "user:pass@tcp(localhost:3306)/myapp?charset=utf8mb4  
db, err := gorm.Open(mysql.Open(dsn), &gorm.Config{})  
gormz.SetDB(db)
```

Using environment variables

.env file:

```
DB_DRIVER=postgres  
DB_HOST=localhost
```

```
DB_PORT=5432
DB_NAME=myapp
DB_USER=postgres
DB_PASSWORD=secret
```

config.go:

```
package config

import (
    "fmt"
    "os"

    "github.com/joho/godotenv"
    "github.com/light-tech-dev/gormz"
    "gorm.io/driver/postgres"
    "gorm.io/driver/sqlite"
    "gorm.io/gorm"
)

func InitDB() error {
    _ = godotenv.Load()

    driver := os.Getenv("DB_DRIVER")
    var dialector gorm.Dialector

    switch driver {
    case "postgres":
        dsn := fmt.Sprintf(
            "host=%s port=%s user=%s password=%s dbname=%s",
            os.Getenv("DB_HOST"),
            os.Getenv("DB_PORT"),
            os.Getenv("DB_USER"),
            os.Getenv("DB_PASSWORD"),
            os.Getenv("DB_NAME"),
        )
        dialector = postgres.Open(dsn)
    case "sqlite":
        dialector = sqlite.Open(os.Getenv("DB_NAME") + ".")
    }
```

```
default:
    return fmt.Errorf("unsupported driver: %s", drive
}

db, err := gorm.Open(dialector, &gorm.Config{})
if err != nil {
    return err
}

gormz.SetDB(db)
return nil
}
```

3. Understanding GORM First

Since gormz is built on GORM, you need to know the basics of GORM. If you already know GORM, **skip this section**.

What is GORM?

GORM is the most popular ORM for Go. It maps Go structs to database tables.

```
type User struct {
    ID      uint
    Name    string
    Email   string
}
```

Becomes:

```
CREATE TABLE users (
    id BIGINT PRIMARY KEY,
    name VARCHAR(255),
```

```
    email VARCHAR(255)
);
```

Basic GORM operations

Create:

```
user := &User{Name: "Ali", Email: "ali@test.com"}
db.Create(user)
// user.ID is populated
```

Read:

```
var user User
db.First(&user, 1) // by ID
db.First(&user, "email = ?", "a@b.com") // by field
db.Where("age > ?", 18).Find(&users) // multiple
```

Update:

```
db.Model(&user).Update("name", "New Name")
db.Model(&user).Updates(map[string]any{"name": "New", "ag
```

Delete:

```
db.Delete(&user, 1)
```

GORM's pain points

1. **No type safety** — `Model(&User{})` repeated everywhere
2. **String-based queries** — `Where("age > ?", 18)` is error-prone
3. **Verbose** — chain of `.Where()` is repetitive
4. **No immutability** — queries share state
5. **Difficult pagination** — manual `Count() + Limit() +`

```
Offset()
```

How gormz improves this

GORM	gormz
<code>db.Model(&User{}).Where("age > ?", 18).Find(&u)</code>	<code>gormz.New[User]().Filter("age__gt", 18).All()</code>
<code>db.First(&u, 1)</code>	<code>gormz.New[User]().Get(1)</code>
No type safety	<code>QuerySet[T]</code> typed
Mutable	Immutable
Manual pagination	<code>.Paginate(1, 20)</code>

GORM features you keep

Since gormz uses GORM under the hood, you keep **everything**:

- ✓ GORM hooks (`BeforeCreate`, `AfterUpdate`, ...)
- ✓ GORM tags (`gorm: "..."`)
- ✓ All databases GORM supports
- ✓ Auto-migrations
- ✓ Soft delete
- ✓ Associations
- ✓ Custom types
- ✓ And more...

gormz just wraps GORM's API with a nicer interface.

4. Your First gormz Query

Let's build a small app step by step.

Step 1 — Define a model

```
package main

import "time"

type User struct {
    ID          uint          `gorm:"primaryKey"`
    Name        string        `gorm:"size:255;not null"`
    Email       string        `gorm:"uniqueIndex;size:255;not null"`
    Age         int           `gorm:"default:0"`
    Active      bool          `gorm:"default:true"`
    CreatedAt   time.Time
    UpdatedAt   time.Time
}

func (User) TableName() string {
    return "users"
}
```

Step 2 — Setup

```
func setup() {
    db, err := gorm.Open(sqlite.Open("app.db"), &gorm.Con
    if err != nil {
        log.Fatal(err)
    }
    gormz.SetDB(db)

    if err := gormz.Migrate[User](); err != nil {
        log.Fatal(err)
    }
}
```

Step 3 — Create records

```
// Single
user := &User{Name: "Ali", Email: "ali@test.com", Age: 30}
err := gormz.New[User]().Create(user)
fmt.Println("Created ID:", user.ID) // 1

// Multiple
users := []User{
    {Name: "Sara", Email: "sara@test.com", Age: 25},
    {Name: "Omar", Email: "omar@test.com", Age: 17},
    {Name: "Layla", Email: "layla@test.com", Age: 35},
}
err = gormz.New[User]().CreateMany(users)
```

Step 4 — Read records

```
// By ID
user, err := gormz.New[User]().Get(1)

// First matching
user, err := gormz.New[User]().
    Filter("email", "ali@test.com").
    First()

// All matching
users, err := gormz.New[User]().
    Filter("active", true).
    Filter("age__gte", 18).
    All()

// Count
count, err := gormz.New[User]().Count()

// Exists
exists, err := gormz.New[User]().
    Filter("email", "ali@test.com").
    Exists()
```

Step 5 — Update records

```
// Single field
err := gormz.New[User]().Update(1, "age", 31)

// Multiple fields
affected, err := gormz.New[User]().
    Filter("age__lt", 18).
    UpdateMany(map[string]any{
        "active": false,
        "status": "minor",
    })
fmt.Println("Updated:", affected)
```

Step 6 — Delete records

```
// Single
err := gormz.New[User]().Delete(1)

// Bulk (requires condition)
affected, err := gormz.New[User]().
    Filter("active", false).
    DeleteMany()
```

Step 7 — Pagination

```
page, err := gormz.New[User]().
    Filter("active", true).
    OrderBy("name").
    Paginate(1, 20)

fmt.Println("Page:", page.Page)
fmt.Println("Total:", page.Total)
fmt.Println("Total pages:", page.TotalPages)
fmt.Println("Items:", len(page.Items))
```

```
for _, u := range page.Items {
    fmt.Println(u.Name)
}
```

Full example

```
package main

import (
    "fmt"
    "log"

    "github.com/light-tech-dev/gormz"
    "gorm.io/driver/sqlite"
    "gorm.io/gorm"
)

type User struct {
    ID      uint   `gorm:"primaryKey"`
    Name    string `gorm:"size:255;not null"`
    Email   string `gorm:"uniqueIndex;size:255;not null"`
    Age     int    `gorm:"default:0"`
    Active  bool   `gorm:"default:true"`
}

func (User) TableName() string { return "users" }

func main() {
    // Setup
    db, _ := gorm.Open(sqlite.Open(":memory:"), &gorm.Con
    gormz.SetDB(db)
    gormz.MustMigrate[User]()

    // Create
    user := &User{Name: "Ali", Email: "ali@test.com", Age
    gormz.New[User]().Create(user)
    fmt.Println("Created:", user.ID)
```

```
// Read
found, _ := gormz.New[User]().Get(user.ID)
fmt.Println("Found:", found.Name)

// Update
gormz.New[User]().Update(user.ID, "age", 31)

// List
users, _ := gormz.New[User]().Filter("active", true).
fmt.Println("Active users:", len(users))

// Delete
gormz.New[User]().Delete(user.ID)
fmt.Println("Done!")
}
```

5. Core Concepts In Depth

Concept 1 — Generic QuerySet[T]

`QuerySet[T]` is the **heart** of gormz. `T` is your model type.

```
users := gormz.New[User]() // QuerySet[User]
orders := gormz.New[Order]() // QuerySet[Order]
products := gormz.New[Product]() // QuerySet[Product]
```

Why it matters:

- **Compile-time checks** — can't mix types
- **Autocomplete** — IDE knows every field
- **No reflection surprises** — types are explicit

Without generics (raw GORM):

```
db.Model(&User{}).Where("age > ?", 18).Find(&users)
//                ^^^ String - no autocomplete, no check
```

With generics (gormz):

```
gormz.New[User]().Filter("age__gt", 18).All()
//                ^^^ Compiler + IDE help
```

Concept 2 — Immutability

Every method returns a **new** `QuerySet`. The original never changes.

```
base := gormz.New[User]().Filter("active", true)

// Create 3 new queries from `base`
adults := base.Filter("age__gte", 18)
young := base.Filter("age__lt", 18)
sorted := base.OrderBy("-created_at")

// `base` is STILL just "active = true"
```

Why it matters: thread safety.

```
base := gormz.New[User]().Filter("active", true)

var wg sync.WaitGroup

for i := 0; i < 10; i++ {
    wg.Add(1)
    go func(id int) {
        defer wg.Done()
        users, _ := base.Filter("id", id).All()
        _ = users
    }(i)
}

wg.Wait()
```

```
// ✓ Safe – no race conditions
```

Concept 3 — Panic vs Error

Panic mode (fast, for trusted input):

```
q := gormz.New[User]().Filter("name", "Ali") // OK
```

Error mode (safe, for external input):

```
q, err := gormz.New[User]().TryFilter(userInput, "Ali")
if err != nil {
    // handle invalid field
}
```

Rule of thumb:

- **Internal code** (hardcoded fields): use `Filter`, `OrderBy`, ...
- **External input** (user, API): use `TryFilter`, `TryOrderBy`, ...

Concept 4 — Method Chaining

Methods chain naturally:

```
users, err := gormz.New[User]().
    Filter("active", true).
    Filter("age__gte", 18).
    Filter("name__icontains", "ali").
    OrderBy("-created_at", "name").
    Limit(20).
    Preload("Orders").
    All()
```

Each method returns a new `QuerySet`, so you can chain forever.

Concept 5 — Context Propagation

Pass a `context.Context` for timeouts, cancellation, and tracing:

```
ctx, cancel := context.WithTimeout(context.Background(),
defer cancel()

users, err := gormz.New[User]().
    WithContext(ctx).
    Filter("active", true).
    All()

if errors.Is(err, context.DeadlineExceeded) {
    log.Println("Query timed out")
}
```

6. Defining Models

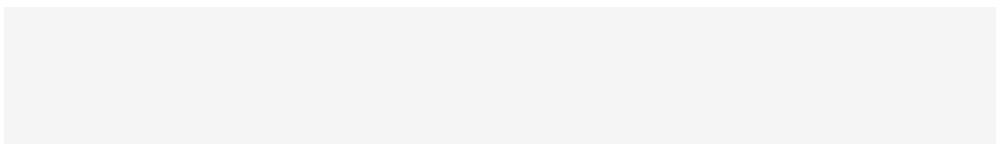
Basic model

```
type User struct {
    ID      uint    `gorm:"primaryKey"`
    Name    string  `gorm:"size:255;not null"`
    Email   string  `gorm:"uniqueIndex;size:255"`
}

func (User) TableName() string { return "users" }
```

With BaseModel

Encapsulate common fields:



```
type BaseModel struct {
    ID          uint      `gorm:"primaryKey" json:"id"`
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`
}

type User struct {
    BaseModel
    Name  string `gorm:"size:255;not null"`
    Email string `gorm:"uniqueIndex;size:255"`
}
```

Common GORM tags

Tag	Purpose
<code>gorm:"primaryKey"</code>	Primary key
<code>gorm:"uniqueIndex"</code>	Unique index
<code>gorm:"index"</code>	Regular index
<code>gorm:"size:255"</code>	VARCHAR(255)
<code>gorm:"not null"</code>	NOT NULL
<code>gorm:"default:0"</code>	Default value
<code>gorm:"type:decimal(10,2)"</code>	Custom SQL type
<code>gorm:"type:text"</code>	TEXT column
<code>gorm:"autoIncrement"</code>	Auto-increment
<code>gorm:"autoCreateTime"</code>	Auto-set on create
<code>gorm:"autoUpdateTime"</code>	Auto-set on update

Soft delete

```
import "gorm.io/gorm"
```

```

type User struct {
    ID          uint
    Name        string
    DeletedAt   gorm.DeletedAt `gorm:"index"`
}

```

Behavior:

- `q.Delete(id)` — soft-deletes
- `q.All()` — excludes soft-deleted
- `q.WithDeleted()` — includes soft-deleted
- `q.OnlyDeleted()` — only soft-deleted
- `q.HardDelete(id)` — permanently removes
- `q.Restore(id)` — undeletes

JSON columns

```

type User struct {
    ID          uint
    Metadata    map[string]any `gorm:"serializer:json"`
}

```

Time columns

```

import "time"

type User struct {
    ID          uint
    CreatedAt   time.Time
    UpdatedAt   time.Time
    DeletedAt   gorm.DeletedAt
    BirthDate   time.Time
    LastLogin   *time.Time    // nullable
}

```

Enums (custom types)

```
type Status string

const (
    StatusActive    Status = "active"
    StatusInactive  Status = "inactive"
    StatusBanned    Status = "banned"
)

type User struct {
    ID      uint
    Status  Status `gorm:"type:varchar(20);default:'active'"`
}
```

Composite indexes

```
type Order struct {
    ID          uint `gorm:"primaryKey"`
    TenantID    uint `gorm:"index:idx_tenant_status,priority:1"`
    Status      string `gorm:"index:idx_tenant_status,priority:2"`
}

// Creates: CREATE INDEX idx_tenant_status ON orders (tenant_id, status)
```

Foreign keys

```
type User struct {
    ID      uint
    Orders []Order `gorm:"foreignKey:UserID"`
}

type Order struct {
    ID      uint
    UserID  uint
    User    *User `gorm:"foreignKey:UserID"`
}
```

```
}
```

Table naming strategies

Option 1 — Explicit:

```
func (User) TableName() string { return "users" }
```

Option 2 — Automatic (default):

```
// User → users  
// SaleOrder → sale_orders
```

Option 3 — Custom naming strategy:

```
db, _ := gorm.Open(sqlite.Open("app.db"), &gorm.Config{  
    NamingStrategy: schema.NamingStrategy{  
        TablePrefix: "app_",  
        SingularTable: true, // user not users  
    },  
})
```

7. Reading Data (Queries)

Getting one record

By ID:

```
user, err := gormz.New[User]().Get(1)
```

Returns `NotFoundError` if not found.

By ID (nil if not found):

```
user, err := gormz.New[User]().GetOrNil(1)
if user == nil {
    // not found
}
```

First matching:

```
user, err := gormz.New[User]().Filter("email", "a@b.com")
```

Returns `NotFoundError` if none.

First matching (nil if none):

```
user, err := gormz.New[User]().Filter("email", "a@b.com")
if user == nil {
    // no match
}
```

By field:

```
user, err := gormz.New[User]().Find("email", "a@b.com")
user, err := gormz.New[User]().FindOrNil("email", "a@b.com")
```

Without ordering:

```
user, err := gormz.New[User]().Take()
```

Getting multiple records

```
users, err := gormz.New[User]().
    Filter("active", true).
    Filter("age__gte", 18).
    OrderBy("-created_at").
    All()
```

Counting

```
count, err := gormz.New[User]().Count()
count, err := gormz.New[User]().Filter("active", true).Co
```

Checking existence

```
exists, err := gormz.New[User]().
    Filter("email", "a@b.com").
    Exists()
```

Extracting one column

```
var emails []string
err := gormz.New[User]().Pluck("email", &emails)
```

Field selection

Select specific fields:

```
users, err := gormz.New[User]().
    Select("id", "name", "email").
    All()
// SELECT id, name, email FROM users
```

Omit fields:

```
users, err := gormz.New[User]().
    Omit("password", "secret_key").
    All()
```

Raw SQL select:

```

var results []struct {
    Status string
    Count  int64
}
err := gormz.New[User]().
    SelectRaw("status", "COUNT(*) as count").
    GroupBy("status").
    ScanInto(&results)

```

Ordering

```

.OrderBy("name")           // name ASC
.OrderBy("-created_at")    // created_at DESC
.OrderBy("status", "-age") // status ASC, age DESC

```

Limits and offsets

```

.Limit(10)           // LIMIT 10
.Offset(20)          // OFFSET 20
.Limit(10).Offset(20) // LIMIT 10 OFFSET 20
.Page(3, 20)         // page 3, 20 per page

```

Complete example

```

// Find top 10 active users in Cairo, ordered by signup d
users, err := gormz.New[User]().
    Filter("active", true).
    Filter("city", "Cairo").
    OrderBy("-created_at").
    Limit(10).
    All()

if err != nil {
    log.Fatal(err)
}

```

```
}

for _, u := range users {
    fmt.Printf("%s (%s)\n", u.Name, u.Email)
}
```

8. Writing Data (Mutations)

Creating records

Single:

```
user := &User{Name: "Ali", Email: "ali@test.com"}
err := gormz.New[User]().Create(user)
fmt.Println(user.ID) // populated
```

Multiple:

```
users := []User{
    {Name: "Ali", Email: "ali@test.com"},
    {Name: "Sara", Email: "sara@test.com"},
}
err := gormz.New[User]().CreateMany(users)
```

In batches (for large sets):

```
users := make([]User, 100000)
// fill users...
err := gormz.New[User]().CreateInBatches(users, 1000)
// Inserts in chunks of 1000
```

Updating records

Single field:

```
err := gormz.New[User]().Update(1, "name", "New Name")
```

Skip hooks:

```
err := gormz.New[User]().UpdateColumn(1, "login_count", 5)
```

Multiple fields (requires condition):

```
affected, err := gormz.New[User]().
    Filter("age__lt", 18).
    UpdateMany(map[string]any{
        "active": false,
        "status": "minor",
    })
```

Upsert (create or update by primary key):

```
user.Name = "Updated Name"
err := gormz.New[User]().Save(user)
```

Deleting records

Soft delete:

```
err := gormz.New[User]().Delete(1)
```

Hard delete:

```
err := gormz.New[User]().HardDelete(1)
```

Bulk delete (requires condition):

```
affected, err := gormz.New[User]().
```

```
Filter("active", false).
DeleteMany()
```

Restore soft-deleted:

```
err := gormz.New[User]().Restore(1)

affected, err := gormz.New[User]().
    Filter("deleted_at__isnull", false).
    RestoreAll()
```

Complete CRUD example

```
func main() {
    // Setup
    db, _ := gorm.Open(sqlite.Open(":memory:"), &gorm.Con
    gormz.SetDB(db)
    gormz.MustMigrate[User]()

    // CREATE
    user := &User{Name: "Ali", Email: "ali@test.com"}
    if err := gormz.New[User]().Create(user); err != nil
        log.Fatal(err)
    }
    fmt.Println("Created:", user.ID)

    // READ
    found, err := gormz.New[User]().Get(user.ID)
    if err != nil {
        log.Fatal(err)
    }
    fmt.Println("Found:", found.Name)

    // UPDATE
    if err := gormz.New[User]().Update(user.ID, "name", "
        log.Fatal(err)
    }
}
```

```

    fmt.Println("Updated")

    // LIST
    users, _ := gormz.New[User]().All()
    fmt.Println("Total:", len(users))

    // DELETE
    if err := gormz.New[User]().Delete(user.ID); err != nil {
        log.Fatal(err)
    }
    fmt.Println("Deleted")
}

```

9. Advanced Lookups — Complete Guide

Advanced lookups are the **most powerful feature** in gormz. They let you express complex conditions in a clean, Django-like syntax:

`field__lookup`.

The syntax

```
field__lookup    →    SQL condition
```

Example:

```

Filter("age__gte", 18)    →    WHERE age >= 18
Filter("name__contains", "ali")    →    WHERE name LIKE '%a
Filter("id__in", []int{1,2,3})    →    WHERE id IN (1,2,3)

```

Complete lookup reference

Equality

```
// field = value
q.Filter("status", "active")

// field != value
q.Filter("status__ne", "deleted")
```

Comparison

```
// >
q.Filter("age__gt", 18)

// >=
q.Filter("age__gte", 18)

// <
q.Filter("price__lt", 100)

// <=
q.Filter("price__lte", 100)
```

String operations

```
// LIKE '%value%' - case sensitive
q.Filter("name__contains", "ali")

// LOWER(field) LIKE '%value%' - case insensitive
q.Filter("name__icontains", "ALI")

// LIKE 'value%'
q.Filter("email__startswith", "admin")

// LOWER LIKE 'value%'
q.Filter("email__istartswith", "ADMIN")

// LIKE '%value%'
q.Filter("email__endswith", "@gmail.com")
```

```
// LOWER LIKE '%value'  
q.Filter("email__iendswith", "@GMAIL.COM")
```

Set operations

```
// IN (1, 2, 3)  
q.Filter("id__in", []int{1, 2, 3})  
  
// NOT IN (4, 5)  
q.Filter("status__notin", []string{"banned", "deleted"})
```

Null checks

```
// IS NULL  
q.Filter("deleted_at__isnull", true)  
  
// IS NOT NULL  
q.Filter("email_verified_at__isnull", false)
```

Range

```
// BETWEEN 100 AND 500  
q.Filter("price__between", []float64{100, 500})  
  
// BETWEEN dates  
q.Filter("created_at__between", []time.Time{start, end})
```

Regex

```
// REGEXP (case sensitive)  
q.Filter("email__regex", "^[a-z]+@")  
  
// LOWER(field) REGEXP (case insensitive)  
q.Filter("phone__iregex", "^\\+20")
```

Date parts

```
// Year
q.Filter("created_at__year", 2025)

// Month
q.Filter("created_at__month", 12)

// Day
q.Filter("created_at__day", 25)
```

Real-world examples

User search:

```
users, _ := gormz.New[User]().
    Filter("active", true).
    Filter("age__gte", 18).
    Filter("age__lte", 65).
    Filter("name__icontains", "ali").
    OrderBy("-created_at").
    All()
```

Product filter:

```
q := gormz.New[Product]().Filter("active", true)

if minPrice > 0 {
    q = q.Filter("price__gte", minPrice)
}
if maxPrice > 0 {
    q = q.Filter("price__lte", maxPrice)
}
if categoryID > 0 {
    q = q.Filter("category_id", categoryID)
}
```

```
if search != "" {
    q = q.Filter("name__icontains", search)
}

products, err := q.OrderBy("-created_at").All()
```

Date range:

```
orders, _ := gormz.New[Order]().
    Filter("created_at__gte", startDate).
    Filter("created_at__lte", endDate).
    Filter("status", "paid").
    All()
```

Conditional filters with FilterIf

For optional filters (like search inputs), use `FilterIf`:

```
q := gormz.New[User]().Filter("active", true)

q = q.FilterIf(search != "", "name__icontains", search)
q = q.FilterIf(minAge > 0, "age__gte", minAge)
q = q.FilterIf(maxAge > 0, "age__lte", maxAge)
q = q.FilterIf(categoryID > 0, "category_id", categoryID)

users, err := q.All()
```

Excluding records

```
// Simple exclusion
q.Exclude("status", "deleted")

// Negated lookup
q.Exclude("id__in", []int{1, 2, 3})
```

10. Q Builder — Complex Conditions

Sometimes you need `(A AND B) OR (C AND D)` — that's what Q Builder is for.

Simple OR

```
q := gormz.Or(  
    gormz.Eq("status", "active"),  
    gormz.Eq("status", "pending"),  
)  
  
users, _ := gormz.New[User]().Q(q).All()  
// WHERE (status = 'active') OR (status = 'pending')
```

Simple AND

```
q := gormz.And(  
    gormz.Eq("active", true),  
    gormz.Gt("age", 18),  
)  
  
users, _ := gormz.New[User]().Q(q).All()  
// WHERE (active = true) AND (age > 18)
```

Nested groups

This is where Q Builder shines:

```
// (active = true AND (age > 18 OR verified = true))  
q := gormz.And(  
    gormz.Eq("active", true),  
    gormz.Or(  
        gormz.Gt("age", 18),  
        gormz.Eq("verified", true),  
    ),  
)
```

```

    gormz.Or(
        gormz.Gt("age", 18),
        gormz.Eq("verified", true),
    ),
)

users, _ := gormz.New[User]() .Q(q) .All()

```

Real-world: search across multiple fields

```

// Search for a term across name, email, or phone
searchQ := gormz.Or(
    gormz.Contains("name", term),
    gormz.Contains("email", term),
    gormz.Contains("phone", term),
)

users, _ := gormz.New[User]() .
    Filter("active", true) .
    Q(searchQ) .
    All()

```

Real-world: complex filter

```

// Users who are:
//   - (admin OR staff)
//   - AND (active OR last_login within 30 days)
//   - AND NOT banned

q := gormz.And(
    gormz.Or(
        gormz.Eq("is_admin", true),
        gormz.Eq("is_staff", true),
    ),
    gormz.Or(
        gormz.Eq("active", true),

```

```

        gormz.Gt("last_login", time.Now().AddDate(0, 0, -
    ),
    gormz.Not(gormz.Eq("status", "banned")),
)

users, _ := gormz.New[User]().Q(q).All()

```

Available Q helpers

```

gormz.Eq(field, value)           // =
gormz.Ne(field, value)           // !=
gormz.Gt(field, value)           // >
gormz.Gte(field, value)          // >=
gormz.Lt(field, value)           // <
gormz.Lte(field, value)          // <=
gormz.Contains(field, value)     // LIKE '%value%'
gormz.StartsWith(field, value)   // LIKE 'value%'
gormz.EndsWith(field, value)     // LIKE '%value'
gormz.In(field, []any{...})      // IN (...)
gormz.IsNull(field)              // IS NULL
gormz.NotNull(field)             // IS NOT NULL
gormz.Raw(sql, args...)          // Raw SQL
gormz.Not(clause)                // NOT (...)

```

Method chaining with Q

```

q := gormz.Qb().
    And(gormz.Eq("a", 1)).
    AndGroup(gormz.Eq("b", 2), gormz.Eq("c", 3))
// (a = 1) AND ((b = 2) AND (c = 3))

q := gormz.Qb().
    And(gormz.Eq("a", 1)).
    OrGroup(gormz.Eq("b", 2), gormz.Eq("c", 3))
// (a = 1) OR ((b = 2) OR (c = 3))

```

Debugging Q

```
q := gormz.Or(
    gormz.Eq("status", "active"),
    gormz.Eq("status", "pending"),
)

sql, args := q.ToSQL()
fmt.Println(sql)      // (status = ?) OR (status = ?)
fmt.Println(args)     // [active pending]
```

11. Pagination

Basic usage

```
page, err := gormz.New[User]().
    Filter("active", true).
    OrderBy("-created_at").
    Paginate(1, 20) // page 1, 20 items per page

// Access
fmt.Println("Items:", len(page.Items))
fmt.Println("Total:", page.Total)
fmt.Println("Page:", page.Page)
fmt.Println("Per page:", page.PerPage)
fmt.Println("Total pages:", page.TotalPages)
fmt.Println("Has next:", page.HasNext)
fmt.Println("Has prev:", page.HasPrev)
```

Available fields

```
type PaginatedResult[T] struct {
```

```

Items      []T    // Current page items
Total      int64  // Total records
Page       int    // Current page number
PerPage    int    // Items per page
TotalPages int    // Total number of pages
HasNext    bool   // Is there a next page?
HasPrev    bool   // Is there a previous page?
}

```

Helper methods

```

page.IsEmpty()           // no items in current page
page.Len()               // number of items in current page

first, ok := page.First() // first item (T, bool)
last, ok  := page.Last()  // last item (T, bool)

page.ForEach(func(i int, u User) {
    fmt.Printf("%d: %s\n", i, u.Name)
})

```

Transform results

Map to another type:

```

names := gormz.MapPage(page, func(u User) string {
    return u.Name
})
// names == []string

```

Filter in-memory:

```

actives := gormz.FilterPage(page, func(u User) bool {
    return u.Active
})
// actives == []User

```

HTTP API example

Handler:

```
func ListUsers(c *fiber.Ctx) error {
    page, _ := strconv.Atoi(c.Query("page", "1"))
    perPage, _ := strconv.Atoi(c.Query("per_page", "20"))

    result, err := gormz.New[User]().
        Filter("active", true).
        OrderBy("-created_at").
        Paginate(page, perPage)
    if err != nil {
        return c.Status(500).JSON(fiber.Map{"error": err.
    }

    return c.JSON(fiber.Map{
        "items":      result.Items,
        "total":      result.Total,
        "page":       result.Page,
        "per_page":   result.PerPage,
        "total_pages": result.TotalPages,
        "has_next":   result.HasNext,
        "has_prev":   result.HasPrev,
    })
}
```

Response:

```
{
  "items": [
    {"id": 1, "name": "Ali", ...},
    {"id": 2, "name": "Sara", ...}
  ],
  "total": 47,
  "page": 1,
  "per_page": 20,
  "total_pages": 3,
```

```
"has_next": true,  
"has_prev": false  
}
```

Cursor-style pagination (manual)

For large datasets, use cursor-based pagination:

```
users, err := gormz.New[User]().  
    Filter("id__gt", lastID). // cursor  
    OrderBy("id").  
    Limit(20).  
    All()
```

Best practices

- ✓ **Always paginate** large results
- ✓ **Default to 20 items** per page
- ✓ **Cap at 100** per page
- ✓ **Order before paginating** (for consistency)
- ✗ **Never** return unpaginated large lists

```
// ✓ Good  
perPage := 20  
if p, _ := strconv.Atoi(c.Query("per_page")); p > 0 && p  
    perPage = p  
}
```

```
// ✗ Bad  
perPage := 1000000 // will load everything
```

12. Relations & Preloading

The N+1 problem

Bad code:

```
users, _ := gormz.New[User]().All()           // 1 query

for _, u := range users {
    orders, _ := gormz.New[Order]().
        Filter("user_id", u.ID).
        All()                                   // N queries
    fmt.Println(u.Name, len(orders))
}
// Total: 1 + N queries
```

Good code:

```
users, _ := gormz.New[User]().Preload("Orders").All() //
for _, u := range users {
    fmt.Println(u.Name, len(u.Orders))
}
// Total: 2 queries
```

Model definitions

Has-many:

```
type User struct {
    ID      uint
    Name    string
    Orders []Order `gorm:"foreignKey:UserID"`
}

type Order struct {
    ID      uint
    UserID  uint
    Total   float64
}
```

Belongs-to:

```
type Order struct {
    ID          uint
    UserID      uint
    User        *User `gorm:"foreignKey:UserID"`
    Total       float64
}
```

Many-to-many:

```
type User struct {
    ID      uint
    Name    string
    Roles   []Role `gorm:"many2many:user_roles;"`
}

type Role struct {
    ID      uint
    Name    string
    Users   []User `gorm:"many2many:user_roles;"`
}
```

Preloading

Single level:

```
users, _ := gormz.New[User]().
    Preload("Orders").
    All()
```

Nested:

```
users, _ := gormz.New[User]().
    Preload("Orders").
    Preload("Orders.Items").
    All()
```

With condition:

```
users, _ := gormz.New[User]() .
    Preload("Orders", "status = ?", "paid") .
    All()
```

Multiple relations:

```
users, _ := gormz.New[User]() .
    Preload("Orders") .
    Preload("Profile") .
    Preload("Roles") .
    All()
```

Complete API example

```
// Model
type User struct {
    ID      uint
    Name    string
    Orders  []Order
    Profile *Profile
}

type Order struct {
    ID      uint
    UserID  uint
    Total   float64
    Items   []OrderItem
}

type OrderItem struct {
    ID          uint
    OrderID     uint
    ProductID   uint
    Quantity    int
    Product     *Product
}
```

```

}

// Get user with all data
user, err := gormz.New[User]().
    Preload("Profile").
    Preload("Orders").
    Preload("Orders.Items").
    Preload("Orders.Items.Product").
    Get(userID)

// Now user.Orders[0].Items[0].Product is loaded

```

Handling circular references

Use pointers to avoid infinite loops in JSON:

```

type User struct {
    ID      uint
    Orders []Order `json:"orders,omitempty"`
}

type Order struct {
    ID      uint
    UserID  uint
    User    *User `json:"user,omitempty"` // pointer - av
}

```

Selective preloading

```

// Only preload for this query
users, _ := gormz.New[User]().Preload("Orders").All()

// Without preload
users, _ := gormz.New[User]().All()
// user.Orders is nil

```

Performance tip

Preload is **2 queries** regardless of record count:

- Query 1: `SELECT * FROM users WHERE ...`
 - Query 2: `SELECT * FROM orders WHERE user_id IN (1,2,3,...)`
-

13. Aggregations

Simple aggregates

```
count, _ := q.Count()           // COUNT(*)
total, _ := q.Sum("amount")     // SUM(amount)
avg, _   := q.Avg("age")        // AVG(age)
min, _   := q.Min("price")      // MIN(price)
max, _   := q.Max("price")      // MAX(price)
```

GroupBy

Simple grouping:

```
type StatusCount struct {
    Status string
    Count  int64
}

var results []StatusCount
err := gormz.New[Order]().
    SelectRaw("status", "COUNT(*) as count").
    GroupBy("status").
    ScanInto(&results)
```

Result:

status	count
pending	12
paid	34
shipped	8

Having clause

```
var results []struct {
    UserID uint
    Count  int64
}

err := gormz.New[Order]().
    SelectRaw("user_id", "COUNT(*) as count").
    GroupBy("user_id").
    Having("COUNT(*) > ?", 5).
    ScanInto(&results)
// Only users with more than 5 orders
```

Advanced aggregations

Use the `advanced` package for a fluent API:

```
import "github.com/light-tech-dev/gormz/advanced"

type UserStats struct {
    UserID          uint
    OrderCount      int64
    TotalAmount     float64
    AvgAmount       float64
}

var stats []UserStats
```

```
err := advanced.GroupBy[Order]("user_id").
    Count("id", "order_count").
    Sum("total", "total_amount").
    Avg("total", "avg_amount").
    Having("COUNT(id) > ?", 5).
    OrderBy("-total_amount").
    ScanInto(&stats)
```

Distinct values

```
// Get all distinct emails
emails, err := advanced.Distinct[User]("email")
// emails == []any{"a@test.com", "b@test.com", ...}

// Count distinct
count, err := advanced.CountDistinctValues[User]("city")
```

Dashboard example

```
func GetDashboardStats(ctx context.Context) (map[string]any, error) {
    q := gormz.New[Order]().WithContext(ctx)

    totalOrders, _ := q.Count()
    pendingOrders, _ := q.Filter("status", "pending").Count()
    paidOrders, _ := q.Filter("status", "paid").Count()

    totalRevenue, _ := q.Filter("status", "paid").Sum("total_amount")
    avgOrderValue, _ := q.Filter("status", "paid").Avg("total_amount")

    return map[string]any{
        "total_orders":    totalOrders,
        "pending_orders":  pendingOrders,
        "paid_orders":     paidOrders,
        "total_revenue":   totalRevenue,
        "avg_order_value": avgOrderValue,
    }, nil
}
```

```
}
```

Time-based aggregation

```
// Orders per day for last 30 days
type DailyCount struct {
    Date    string
    Count   int64
    Total   float64
}

var results []DailyCount
err := gormz.New[Order]().
    SelectRaw(
        "DATE(created_at) as date",
        "COUNT(*) as count",
        "SUM(total) as total",
    ).
    Filter("created_at__gte", time.Now().AddDate(0, 0, -30)).
    GroupBy("DATE(created_at)").
    OrderBy("date").
    ScanInto(&results)
```

14. Context & Cancellation

Why context matters

Context allows:

- 🕒 **Timeouts** — cancel long queries
- **Cancellation** — stop on client disconnect
- 🔍 **Tracing** — propagate request IDs
- 🌐 **HTTP integration** — auto-cancel when request ends

Basic timeout

```
ctx, cancel := context.WithTimeout(context.Background(),
defer cancel()

users, err := gormz.New[User]().
    WithContext(ctx).
    Filter("active", true).
    All()

if errors.Is(err, context.DeadlineExceeded) {
    log.Println("Query took too long")
}
```

With HTTP request

Fiber:

```
func GetUsers(c *fiber.Ctx) error {
    users, err := gormz.New[User]().
        WithContext(c.Context()).
        Filter("active", true).
        All()

    if err != nil {
        return c.Status(500).JSON(fiber.Map{"error": err.
    }
    return c.JSON(users)
}
```

Gin:

```
func GetUsers(c *gin.Context) {
    users, err := gormz.New[User]().
        WithContext(c.Request.Context()).
        Filter("active", true).
```

```
        All()
    // ...
}
```

Echo:

```
func GetUsers(c echo.Context) error {
    users, err := gormz.New[User]().
        WithContext(c.Request().Context()).
        Filter("active", true).
        All()
    // ...
}
```

Cancellation

```
ctx, cancel := context.WithCancel(context.Background())
defer cancel()

// Cancel after 1 second
go func() {
    time.Sleep(1 * time.Second)
    cancel()
}()

users, err := gormz.New[User]().
    WithContext(ctx).
    All()

if errors.Is(err, context.Canceled) {
    log.Println("Query cancelled")
}
```

Context storage (advanced)

Store DB instance in context for later retrieval:

```
// Setup: inject instance
ctx := gormz.WithDB(context.Background(), instance)
ctx = gormz.WithGormDB(ctx, db) // or with raw *gorm.DB

// Retrieve anywhere
users, _ := gormz.FromContext[User](ctx).All()
```

Use case: middleware

```
func DBMiddleware(instance *gormz.Instance) fiber.Handler {
    return func(c *fiber.Ctx) error {
        ctx := gormz.WithDB(c.Context(), instance)
        c.SetUserContext(ctx)
        return c.Next()
    }
}

// Later in handler
func GetUsers(c *fiber.Ctx) error {
    users, _ := gormz.FromContext[User](c.Context()).All()
    return c.JSON(users)
}
```

Available functions

<code>gormz.WithDB(ctx, instance)</code>	<code>// store instance</code>
<code>gormz.WithGormDB(ctx, db)</code>	<code>// store *gorm.DB</code>
<code>gormz.DBFromContext(ctx)</code>	<code>// (*Instance, bool)</code>
<code>gormz.FromContext[T](ctx)</code>	<code>// *QuerySet[T]</code>
<code>gormz.MustFromContext[T](ctx)</code>	<code>// *QuerySet[T]</code>

Best practices

- ✓ Always pass context for HTTP handlers
- ✓ Set timeouts for external queries
- ✓ Use `context.Background()` only in tests/scripts

- ✘ Never ignore context cancellation
-

15. Multiple Databases

Why multiple databases?

- **Read/write split** — main + replicas
- **Multi-tenancy** — one DB per tenant
- **Microservices** — different DBs per service
- **Testing** — isolated test databases

Creating an Instance

```
// Write (master) connection
writeDB, _ := gorm.Open(postgres.Open(masterDSN), &gorm.C
writeInstance := gormz.NewInstance(writeDB)

// Read (replica) connection
readDB, _ := gorm.Open(postgres.Open(replicaDSN), &gorm.C
readInstance := gormz.NewInstance(readDB)
```

Querying an Instance

```
// Write to master
err := gormz.QueryOn[User](writeInstance).Create(&user)

// Read from replica
users, _ := gormz.QueryOn[User](readInstance).
    Filter("active", true).
    All()
```

Transactions on Instance

```
err := writeInstance.Transaction(ctx, func(tx *gorm.DB) e
    if err := tx.Create(&user).Error; err != nil {
        return err
    }
    return tx.Create(&order).Error
})
```

Multi-tenant example

```
type TenantManager struct {
    instances map[uint]*gormz.Instance
    mu        sync.RWMutex
}

func NewTenantManager() *TenantManager {
    return &TenantManager{
        instances: make(map[uint]*gormz.Instance),
    }
}

func (tm *TenantManager) Get(tenantID uint) (*gormz.Insta
    tm.mu.RLock()
    if inst, ok := tm.instances[tenantID]; ok {
        tm.mu.RUnlock()
        return inst, nil
    }
    tm.mu.RUnlock()

    // Create connection
    dsn := fmt.Sprintf("tenant_%d.db", tenantID)
    db, err := gorm.Open(sqlite.Open(dsn), &gorm.Config{})
    if err != nil {
        return nil, err
    }
}
```

```

    inst := gormz.NewInstance(db)

    tm.mu.Lock()
    tm.instances[tenantID] = inst
    tm.mu.Unlock()

    return inst, nil
}

```

Usage:

```

func GetUsers(c *fiber.Ctx) error {
    tenantID, _ := strconv.Atoi(c.Get("X-Tenant-ID"))

    inst, err := tenantManager.Get(uint(tenantID))
    if err != nil {
        return c.Status(500).JSON(fiber.Map{"error": err.
    }

    users, err := gormz.QueryOn[User](inst).
        Filter("active", true).
        All()

    return c.JSON(users)
}

```

Testing with Instance

```

func TestUserService(t *testing.T) {
    // Fresh in-memory DB for each test
    db, _ := gorm.Open(sqlite.Open(":memory:"), &gorm.Con
    db.AutoMigrate(&User{})

    instance := gormz.NewInstance(db)

    // Use it
    err := gormz.QueryOn[User](instance).Create(&User{Nam

```

```

require.NoError(t, err)

count, _ := gormz.QueryOn[User](instance).Count()
assert.Equal(t, int64(1), count)
}

```

Global vs Instance

Feature	Global (gormz.New[T])	Instance (gormz.QueryOn[T] (inst))
Setup	gormz.SetDB(db)	gormz.NewInstance(db)
Usage	gormz.New[User] ()	gormz.QueryOn[User] (inst)
Use case	Single DB apps	Multi-DB apps

Available API

```

gormz.NewInstance(db)           // *Instance
instance.DB()                   // *gorm.DB
instance.Ping()                  // error
instance.Close()                 // error
instance.Configure(cfg)          // error
instance.Transaction(ctx, fn)    // error
instance.Migrate(models...)      // error

gormz.GlobalInstance()           // *Instance
gormz.NewWith[T](instance)       // *QuerySet[T]
gormz.QueryOn[T](instance)       // *QuerySet[T]
gormz.QueryOnWithContext[T](i, ctx) // *QuerySet[T]

```

16. Model Registry

The import cycle problem

Imagine two packages that need each other:

```
package user → needs Order
package order → needs User
```

This creates an **import cycle** — Go won't compile it.

The registry solution

Register models once, retrieve them from anywhere:

```
// package user
package user

import "github.com/light-tech-dev/gormz"

type User struct { /* ... */ }

var Users = gormz.Register[User]("user")
```

```
// package order
package order

import "github.com/light-tech-dev/gormz"

type Order struct { /* ... */ }

var Orders = gormz.Register[Order]("order")
```

```
// main package
import (
    "yourapp/user"
    "yourapp/order"
)
```

```
)

user.Users.Filter("active", true).All()
order.Orders.Filter("status", "paid").All()
```

No import cycle!

Complete API

Register:

```
Users := gormz.Register[User]("user") // panics on duplicate
q, err := gormz.TryRegister[User]("user") // returns error
```

Lookup:

```
q, ok := gormz.Lookup[User]("user") // (*QuerySet[User])
q := gormz.MustLookup[User]("user") // panics if missing
exists := gormz.Has("user") // bool
```

List:

```
names := gormz.RegisteredNames() // []string
count := gormz.RegisteredCount() // int
```

Remove:

```
gormz.Unregister("user")
gormz.ClearRegistry() // clear all (types)
```

Bulk:

```
entries := map[string]any{
    "user": gormz.New[User](),
    "order": gormz.New[Order](),
}
```

```
err := gormz.RegisterBatch(entries)

count := gormz.UnregisterBatch("user", "order")
```

Type-based lookup

Find registered models by Go type:

```
hasUser := gormz.HasType[User]()
q, ok := gormz.LookupByType[User]()
q := gormz.MustLookupByType[User]()
names := gormz.NamesByType[User]() // all registered names
```

Snapshot (for testing)

```
// Save current state
snap := gormz.TakeSnapshot()

// Modify registry (add models, remove)
// ...

// Restore snapshot
snap.Restore()

// Or merge (only adds new ones)
count := snap.Merge()
```

Real-world: modular app

```
myapp/
├─ internal/
│   └─ users/
│       ├── model.go    // var Users = gormz.Register[User]
│       └─ service.go   // uses users.Users
```

```

|   |─ orders/
|   |   |─ model.go      // var Orders = gormz.Register[O
|   |   |─ service.go    // uses orders.Orders
|   |─ products/
|   |   |─ model.go
|   |   |─ service.go

```

Each module owns its `QuerySet`, and other modules use it without importing.

17. Transactions In Depth

Why transactions?

A transaction ensures **all-or-nothing** operations:

```

// ✗ Without transaction
db.Create(&order)           // succeeds
db.Update(&user, ...)       // fails
// Now order exists but user wasn't updated – inconsistent

```

Simple transaction

```

err := gormz.Transaction(ctx, func(tx *gorm.DB) error {
    if err := tx.Create(&user).Error; err != nil {
        return err    // triggers rollback
    }
    if err := tx.Create(&order).Error; err != nil {
        return err    // triggers rollback
    }
    return nil        // triggers commit
})

```

Advanced transaction (with retry)

```
import "github.com/light-tech-dev/gormz/advanced"

err := advanced.WithTransaction(ctx, advanced.DefaultTxCo
    func(tx *advanced.Tx) error {
        if err := advanced.Query[User](tx).Create(&user);
            return err
        }
        return advanced.Query[Order](tx).Create(&order)
    })
```

Custom configuration

```
cfg := advanced.TxConfig{
    Isolation:  advanced.IsolationSerializable,
    ReadOnly:   false,
    Timeout:    30 * time.Second,
    Retries:    3,
    RetryDelay: 100 * time.Millisecond,
}

err := advanced.WithTransaction(ctx, cfg, func(tx *advanc
    // ...
})
```

Isolation levels

Level	Description
IsolationReadUncommitted	Dirty reads allowed
IsolationReadCommitted	No dirty reads (Postgres default)
IsolationRepeatableRead	No non-repeatable reads
IsolationSerializable	Full isolation

Manual transaction

For fine control:

```
tx, err := advanced.Begin(ctx, advanced.DefaultTxConfig())
if err != nil {
    return err
}
defer tx.RollbackIfActive() // safe no-op if committed

if err := advanced.Query[User](tx).Create(&user); err != nil {
    return err // deferred rollback triggers
}

return tx.Commit()
```

Nested transactions (savepoints)

```
err := advanced.WithTransaction(ctx, advanced.DefaultTxConfig())
func(tx *advanced.Tx) error {
    // Main operation
    if err := advanced.Query[User](tx).Create(&user); err != nil {
        return err
    }

    // Nested operation – can fail without failing the whole transaction
    err := tx.Nested(func(inner *advanced.Tx) error {
        return advanced.Query[Order](inner).Create(&order)
    })
    if err != nil {
        log.Println("Nested failed:", err)
        // outer continues
    }

    return nil
}
```

Real-world: order processing

```
func (s *OrderService) CreateOrder(ctx context.Context, r
    var order *Order

    err := advanced.WithTransaction(ctx, advanced.Default
        func(tx *advanced.Tx) error {
            // 1. Load products and check stock
            var total float64
            items := make([]OrderItem, 0, len(req.Items))

            for _, itemReq := range req.Items {
                product, err := advanced.Query[Product](t
                if err != nil {
                    return fmt.Errorf("product %d not fou
                }

                if product.Stock < itemReq.Quantity {
                    return fmt.Errorf("insufficient stock
                }

                subtotal := product.Price * float64(itemR
                total += subtotal

                items = append(items, OrderItem{
                    ProductID: product.ID,
                    Quantity:   itemReq.Quantity,
                    UnitPrice:  product.Price,
                    Subtotal:   subtotal,
                })

                // Reduce stock
                product.Stock -= itemReq.Quantity
                if err := advanced.Query[Product](tx).Sav
                    return err
                }
            }

            // 2. Create order
```

```

        order = &Order{
            OrderNumber: generateOrderNumber(),
            Status:      OrderStatusPending,
            TotalAmount: total,
            UserID:      req.UserID,
        }

        if err := advanced.Query[Order](tx).Create(order); err != nil {
            return err
        }

        // 3. Create order items
        for i := range items {
            items[i].OrderID = order.ID
        }

        return advanced.Query[OrderItem](tx).CreateMany(items)
    })

    return order, err
}

```

Best practices

- ✓ Keep transactions short
- ✓ Use `RollbackIfActive()` in `defer`
- ✓ Return errors immediately
- ✓ Use savepoints for nested logic
- ✓ Consider isolation levels
- ✓ Add retry for deadlock-prone operations
- ✗ Don't do HTTP calls inside transactions
- ✗ Don't hold locks longer than needed

18. Bulk Operations

Why bulk operations?

For 100,000 records:

Method	Time
Loop with <code>Create</code>	~500 seconds
<code>CreateInBatches (1000)</code>	~2 seconds
<code>BulkInsert</code>	~1.5 seconds

Bulk insert

```
import "github.com/light-tech-dev/gormz/advanced"

users := make([]User, 100000)
// fill users...

err := advanced.BulkInsert[User](ctx, users, advanced.BulkInsertOptions{
    BatchSize: 1000,
})
```

Bulk upsert

Insert or update on conflict:

```
err := advanced.BulkUpsert[User](ctx, users, advanced.BulkUpsertOptions{
    ConflictColumns: []string{"email"},
    UpdateColumns:   []string{"name", "age"},
})
// INSERT ... ON CONFLICT (email) DO UPDATE SET name=...,
```

Ignore conflicts:

```
err := advanced.BulkUpsert[User](ctx, users, advanced.BulkUpsertOptions{
    ConflictColumns: []string{"email"},
    IgnoreConflicts: true,
})
```

```
        IgnoreOnConflict: true,
    })
    // INSERT ... ON CONFLICT (email) DO NOTHING
```

Bulk update (different values per row)

```
updates := []advanced.UpdateItem[User]{
    {ID: 1, Values: map[string]any{"age": 31, "status": "active"}},
    {ID: 2, Values: map[string]any{"age": 26, "status": "inactive"}},
    {ID: 3, Values: map[string]any{"age": 45, "status": "active"}},
}

err := advanced.BulkUpdate[User](ctx, "id", updates)
// Generates a single UPDATE with CASE WHEN
```

Bulk delete

By IDs:

```
deleted, err := advanced.BulkDeleteByIDs[User](ctx, ids,
```

By condition:

```
deleted, err := advanced.BulkDeleteWhere[User](ctx, "age < ?", cutoff)

deleted, err := advanced.BulkDeleteWhere[User](ctx,
    "status = ? AND created_at < ?",
    "inactive", cutoff)
```

Bulk count

```
count, err := advanced.BulkCount[User](ctx, "active = ?", cutoff)
```

Bulk config

```
type BulkConfig struct {
    BatchSize          int          // records per batch
    UpdateOnConflict   bool          // upsert behavior
    ConflictColumns    []string     // columns to detect conflict
    UpdateColumns       []string     // columns to update on conflict
    IgnoreOnConflict   bool          // skip on conflict
}

func DefaultBulkConfig() BulkConfig {
    return BulkConfig{BatchSize: 1000}
}
```

Real-world: CSV import

```
func ImportUsers(ctx context.Context, r io.Reader) error {
    reader := csv.NewReader(r)
    reader.Read() // skip header

    var users []User
    for {
        record, err := reader.Read()
        if err == io.EOF {
            break
        }
        if err != nil {
            return err
        }

        users = append(users, User{
            Name:  record[0],
            Email: record[1],
            Age:  atoi(record[2]),
        })
    }
}
```

```

        return advanced.BulkInsert[User](ctx, users, advanced
            BatchSize: 1000,
        })
    }

```

Real-world: sync from external API

```

func SyncProducts(ctx context.Context, external []External
    products := make([]Product, len(external))
    for i, p := range external {
        products[i] = Product{
            ExternalID: p.ID,
            Name:        p.Name,
            Price:       p.Price,
        }
    }

    // Upsert by external_id
    return advanced.BulkUpsert[Product](ctx, products, ad
        BatchSize:        500,
        ConflictColumns: []string{"external_id"},
        UpdateColumns:    []string{"name", "price"},
    })
}

```

Best practices

- ✓ Use `BatchSize: 1000` as default
- ✓ Use `BulkUpsert` for sync operations
- ✓ Use `BulkDeleteByIDs` for cleanup
- ⚠ SQLite has 999 parameter limit — smaller batches
- ⚠ PostgreSQL has 65535 parameter limit
- ✗ Don't use bulk for single records

19. CTEs (Common Table Expressions)

What are CTEs?

CTEs are named subqueries that you can reference multiple times. They make complex queries readable.

Without CTE:

```
SELECT * FROM orders
WHERE user_id IN (
    SELECT id FROM users
    WHERE active = true AND created_at > '2025-01-01'
);
```

With CTE:

```
WITH active_users AS (
    SELECT id FROM users
    WHERE active = true AND created_at > '2025-01-01'
)
SELECT * FROM orders WHERE user_id IN (SELECT id FROM act
```

Simple CTE

```
import "github.com/light-tech-dev/gormz/advanced"

activeUsers := advanced.NewCTE("active_users",
    gormz.New[User]().
        Select("id").
        Filter("active", true).
        Filter("created_at__gte", time.Now().AddDate(0, -
orders, err := advanced.With[Order](activeUsers).
    Query(gormz.New[Order]()).
        Where("user_id IN (SELECT id FROM active_users)")
    All()
```

Multiple CTEs

```
activeUsers := advanced.NewCTE("active_users",
    gormz.New[User]().Filter("active", true))

recentOrders := advanced.NewCTE("recent_orders",
    gormz.New[Order]().
        Filter("created_at__gte", time.Now().AddDate(0, 0, 0)))

results, err := advanced.With[User](activeUsers, recentOrders).
    Query(gormz.New[User]().
        Where("id IN (SELECT user_id FROM recent_orders)")
        All())
```

Recursive CTE (tree structure)

Perfect for hierarchical data (categories, org charts, comments):

```
type Category struct {
    ID          uint
    Name        string
    ParentID    *uint
}

// Base query: start from root
base := gormz.New[Category]().Filter("id", rootID)

// Build recursive CTE
tree := advanced.NewRecursiveCTE[Category]("tree", base)

// Add recursive part: children of tree nodes
tree.UnionRaw(
    "SELECT c.* FROM categories c JOIN tree t ON c.parent = t.id"
)

// Execute
results, err := advanced.With[Category](tree).
```

```
Query(gormz.New[Category]()).  
All()
```

Generated SQL:

```
WITH RECURSIVE tree AS (  
    SELECT * FROM categories WHERE id = ?  
    UNION ALL  
    SELECT c.* FROM categories c JOIN tree t ON c.parent_  
)  
SELECT * FROM categories
```

Real-world: comment threads

```
type Comment struct {  
    ID          uint  
    PostID      uint  
    ParentID    *uint  
    Body        string  
    Author      string  
}  
  
func GetCommentThread(ctx context.Context, postID uint) (  
    base := gormz.New[Comment]().  
        Filter("post_id", postID).  
        Filter("parent_id__isnull", true)  
  
    tree := advanced.NewRecursiveCTE[Comment]("thread", b  
    tree.UnionRow(  
        "SELECT c.* FROM comments c JOIN thread t ON c.pa  
    )  
  
    return advanced.With[Comment](tree).  
        Query(gormz.New[Comment]().Filter("post_id", post  
        All()  
}
```

20. Window Functions

What are window functions?

Window functions compute values across a set of rows **without collapsing them** (unlike GROUP BY).

Example: Rank users by salary within each department.

ROW_NUMBER — sequential numbering

```
results, err := gormz.New[User]().
    Select("id", "name", "department_id").
    Window(advanced.RowNumber("rn", "department_id")).
    All()
```

Generated SQL:

```
SELECT id, name, department_id,
       ROW_NUMBER() OVER (PARTITION BY department_id) AS
FROM users
```

RANK — with ties

```
gormz.New[User]().
    Window(advanced.Rank("rnk", "salary DESC", "department_id", "salary").
// RANK() OVER (PARTITION BY department_id ORDER BY salary)
```

LAG / LEAD — previous/next row

```
gormz.New[Order]().
```

```
Select("id", "user_id", "total").
Window(advanced.Lag("total", 1, "prev_total", "user_id"),
Window(advanced.Lead("total", 1, "next_total", "user_id"),
All()
```

Use case: Calculate change between consecutive orders.

Running aggregates

```
// Running total of amounts over time
gormz.New[Order]().
    Select("id", "created_at", "amount").
    Window(advanced.RunningSum("amount", "running_total",
    All())
// SUM(amount) OVER (ORDER BY created_at) AS running_total

// Running average
gormz.New[Order]().
    Window(advanced.RunningAvg("amount", "running_avg", "amount")
```

NTile — percentile buckets

```
// Divide users into 4 quartiles by salary
gormz.New[User]().
    Window(advanced.NTile(4, "quartile", "salary DESC"))
```

FirstValue / LastValue

```
gormz.New[User]().
    Window(advanced.FirstValue("salary", "highest_salary"),
    Window(advanced.LastValue("salary", "lowest_salary"),
```

PercentRank / CumeDist

```
gormz.New[User]() .
    Window(advanced.PercentRank("pct_rank", "salary DESC"
    // PERCENT_RANK() OVER (ORDER BY salary DESC)

gormz.New[User]() .
    Window(advanced.CumeDist("cum_dist", "salary DESC"))
```

Complete list of window functions

```
advanced.RowNumber(alias, partition...)
advanced.Rank(alias, orderBy, partition...)
advanced.DenseRank(alias, orderBy, partition...)
advanced.Lag(field, offset, alias, partition...)
advanced.Lead(field, offset, alias, partition...)
advanced.RunningSum(field, alias, orderBy, partition...)
advanced.RunningCount(alias, orderBy, partition...)
advanced.RunningAvg(field, alias, orderBy, partition...)
advanced.SumOver(field, alias, partition...)
advanced.CountOver(alias, partition...)
advanced.AvgOver(field, alias, partition...)
advanced.NTile(n, alias, orderBy, partition...)
advanced.FirstValue(field, alias, orderBy, partition...)
advanced.LastValue(field, alias, orderBy, partition...)
advanced.NthValue(field, n, alias, orderBy, partition...)
advanced.PercentRank(alias, orderBy, partition...)
advanced.CumeDist(alias, orderBy, partition...)
```

Real-world: top 3 orders per user

```
type RankedOrder struct {
    ID          uint
    UserID      uint
    Total       float64
    Rank        int
}
```

```

var results []RankedOrder
err := gormz.New[Order]().
    Select("id", "user_id", "total").
    Window(advanced.RowNumber("rn", "user_id")).
    Filter("rn__lte", 3). // not valid - need raw
    All()

```

Since filtering window results requires a subquery, use:

```

sql := `
    SELECT * FROM (
        SELECT id, user_id, total,
            ROW_NUMBER() OVER (PARTITION BY user_id OR
        FROM orders
    ) t WHERE rn <= 3
`
db.Raw(sql).Scan(&results)

```

21. Subqueries

Simple subquery

```

// Find users who have at least one paid order
subq := advanced.SubFrom[Order](
    gormz.New[Order]().
        Select("user_id").
        Filter("status", "paid"),
    "user_id",
)

users, err := gormz.New[User]().
    Q(advanced.In("id", subq)).
    All()

```

Generated SQL:

```
SELECT * FROM users
WHERE id IN (SELECT user_id FROM orders WHERE status = 'p
```

NOT IN subquery

```
// Users who have NO orders
users, err := gormz.New[User]().
    Q(advanced.NotIn("id", subq)).
    All()
```

EXISTS subquery

```
users, err := gormz.New[User]().
    Q(advanced.Exists(subq)).
    All()
// WHERE EXISTS (subquery)
```

Comparison subqueries

```
// Users whose salary > average
avgSalary := advanced.SubRaw("SELECT AVG(salary) FROM use

users, err := gormz.New[User]().
    Q(advanced.GtSub("salary", avgSalary)).
    All()
// WHERE salary > (SELECT AVG(salary) FROM users)
```

Subquery operators

```
advanced.In(field, subq) // field IN (subquery)
```

```

advanced.NotIn(field, subq)           // field NOT IN (subquery)
advanced.Exists(subq)                 // EXISTS (subquery)
advanced.NotExists(subq)              // NOT EXISTS (subquery)
advanced.GtSub(field, subq)           // field > (subquery)
advanced.LtSub(field, subq)           // field < (subquery)
advanced.EqSub(field, subq)           // field = (subquery)
advanced.GteSub(field, subq)          // field >= (subquery)
advanced.LteSub(field, subq)          // field <= (subquery)

```

Correlated subqueries

Subqueries that reference the outer query:

```

corr := advanced.NewCorrelated(
    "SELECT COUNT(*) FROM orders WHERE orders.user_id = u
")

users, err := gormz.New[User]().
    Select("id", "name").
    Where("(" + corr.SQL + ") > ?", 5).
    All()

```

Real-world: dashboard query

```

// Find users with high-value orders
bigSpenders := advanced.SubFrom[Order](
    gormz.New[Order]().
        Select("user_id").
        Filter("status", "paid").
        Filter("total__gte", 1000),
    "user_id",
)

users, err := gormz.New[User]().
    Q(advanced.In("id", bigSpenders)).
    Filter("active", true).

```

```
Preload("Orders").  
All()
```

22. Unions

UNION (removes duplicates)

```
q1 := gormz.New[User]().Filter("status", "active")  
q2 := gormz.New[User]().Filter("status", "pending")  
  
users, err := advanced.Union(q1, q2).All()  
// SELECT * FROM users WHERE status = 'active'  
// UNION  
// SELECT * FROM users WHERE status = 'pending'
```

UNION ALL (keeps duplicates)

```
users, err := advanced.UnionAll(q1, q2).All()
```

With OrderBy and Limit

```
users, err := advanced.Union(q1, q2).  
    OrderBy("-created_at").  
    Limit(10).  
    All()
```

Multiple unions

```
q1 := gormz.New[User]().Filter("age__lt", 18)
```

```
q2 := gormz.New[User]().Filter("age__between", []int{18,
q3 := gormz.New[User]().Filter("age__gt", 65)

results, err := advanced.UnionAll(q1, q2, q3).All()
```

Real-world: search across tables

```
// Search users and products for a term
userQ := gormz.New[User]().
    Select("id", "name", "email as description").
    Filter("name__icontains", term)

productQ := gormz.New[Product]().
    Select("id", "name", "description").
    Filter("name__icontains", term)

// Note: both queries must have same column count/types
results, err := advanced.UnionAll(userQ, productQ).All()
```

⚠ **Requirement:** All queries in a union must have the same number of columns with compatible types.

23. Locking — Pessimistic & Optimistic

Two types of locking

Type	When to use
Pessimistic	High contention; lock the row while processing
Optimistic	Low contention; use version field to detect conflicts

Pessimistic locking

Lock a row so no one else can modify it:

```
err := advanced.WithTransaction(ctx, advanced.DefaultTxCo
func(tx *advanced.Tx) error {
    // Lock user row for update
    user, err := advanced.WithLock[User](tx.Query[Use
        ForUpdate().
        Get(1)
    if err != nil {
        return err
    }

    // Safe to modify – no one else can touch this ro
    user.Balance -= 100
    return tx.Query[User]().Save(user)
})
```

Lock modes

```
// Exclusive – blocks all other locks
.ForUpdate()
// FOR UPDATE

// Shared – allows concurrent read-locks
.ForShare()
// FOR SHARE

// PostgreSQL-specific
.ForNoKeyUpdate()
.ForKeyShare()
```

Optimistic locking

Add a `version` field to your model:

```
type Order struct {
```

```

    ID      uint
    Version int64 `gorm:"default:1"`
    Status   string
    Total    float64
}

```

Update with version check:

```

err := advanced.WithOptimisticLock[Order](gormz.New[Order]().
    UpdateIfVersion(ctx, orderID, currentVersion, map[string]string{
        "status": "paid",
    })

if err != nil {
    // version mismatch - someone else updated it
    return errors.New("order was modified by another process")
}

```

Real-world: bank transfer

```

func TransferMoney(ctx context.Context, fromID, toID uint) error {
    return advanced.WithTransaction(ctx, advanced.DefaultTransactionFunc(
        func(tx *advanced.Tx) error {
            // Lock both accounts (in ID order to prevent deadlock)
            first, second := fromID, toID
            if first > second {
                first, second = second, first
            }

            _, err := advanced.WithLock[Account](tx.Query().
                ForUpdate().
                Get(first))
            if err != nil {
                return err
            }

            _, err = advanced.WithLock[Account](tx.Query().
                ForUpdate().
                Get(second))
            if err != nil {
                return err
            }

            // Perform the transfer
            from := tx.Query().Get(first)
            to := tx.Query().Get(second)
            if from.Balance < 0 || to.Balance > 0 {
                return errors.New("invalid balance")
            }
            from.Balance -= amount
            to.Balance += amount
            tx.Save(from)
            tx.Save(to)
        })
}

```

```

        ForUpdate().
        Get(second)
    if err != nil {
        return err
    }

    // Load both
    from, _ := tx.Query[Account]().Get(fromID)
    to, _ := tx.Query[Account]().Get(toID)

    if from.Balance < amount {
        return errors.New("insufficient funds")
    }

    from.Balance -= amount
    to.Balance += amount

    if err := tx.Query[Account]().Save(from); err != nil {
        return err
    }
    return tx.Query[Account]().Save(to)
})
}

```

Locking API

```

// Pessimistic
advanced.WithLock[T](q) *PessimisticQuery[T]
    .ForUpdate()
    .ForShare()
    .ForNoKeyUpdate() // PostgreSQL
    .ForKeyShare()    // PostgreSQL
    .Filter(field, value)
    .Where(sql, args...)
    .All()
    .First()
    .Get(id)

```

```
// Optimistic
advanced.WithOptimisticLock[T](q) *OptimisticQuery[T]
    .Field("custom_version_field") // default: "version"
    .UpdateIfVersion(ctx, id, version, updates)
    .DeleteIfVersion(ctx, id, version)
    .IncrementVersion(ctx, id)

// Helpers
advanced.LockClause(mode) clause.Locking
advanced.LockClauseWithTable(mode, table)
```

Best practices

- ✓ **Pessimistic** for bank transfers, inventory updates
 - ✓ **Optimistic** for user profiles, editable documents
 - ✓ **Lock in consistent order** to prevent deadlocks
 - ✓ **Keep locks short** — commit ASAP
 - ✗ **Never** hold locks during external API calls
-

24. Retry with Backoff

Why retry?

Some errors are transient:

- **Deadlocks** — another transaction blocked us
- **Serialization failures** — concurrent modification
- **Connection resets** — network blip
- **Lock timeouts** — resource temporarily busy

Simple retry

```
err := advanced.Retry(ctx, advanced.DefaultRetryConfig(),
```

```
    return gormz.New[User]().Create(&user)
})
```

Retry with custom config

```
cfg := advanced.RetryConfig{
    MaxAttempts: 5,
    InitialDelay: 100 * time.Millisecond,
    MaxDelay: 5 * time.Second,
    Multiplier: 2.0,
    Jitter: 0.1,
    RetryIf: advanced.IsRetryableError,
}

err := advanced.Retry(ctx, cfg, func() error {
    return doSomethingRisky()
})
```

Backoff strategies

Exponential:

```
backoff := advanced.ExponentialBackoff(100*time.Millisecond, 5
// 100ms, 200ms, 400ms, 800ms, ..., up to 10s
```

Linear:

```
backoff := advanced.LinearBackoff(100*time.Millisecond, 5
// 100ms, 200ms, 300ms, ..., up to 5s
```

Constant:

```
backoff := advanced.ConstantBackoff(500 * time.Millisecond, 5
// 500ms, 500ms, 500ms, ...
```

Retry with backoff

```
backoff := advanced.ExponentialBackoff(100*time.Millisecond)

err := advanced.RetryWithBackoff(ctx, 5, backoff, func()
    return gormz.New[User]().Create(&user)
})
```

Retry DB operations

```
// Convenient wrapper for DB operations
err := advanced.RetryDB(ctx, func(db *gorm.DB) error {
    return db.Create(&user).Error
})
```

What gets retried automatically?

IsRetryableError checks for:

- deadlock
- lock wait timeout
- could not serialize
- serialization failure
- database is locked (SQLite)
- connection reset
- connection refused
- broken pipe
- too many connections
- server closed the connection
- i/o timeout

Real-world: order processing with retry

```
func (s *OrderService) CreateOrder(ctx context.Context, r
    var order *Order
```

```

    err := advanced.RetryDB(ctx, func(db *gorm.DB) error {
        return db.Transaction(func(tx *gorm.DB) error {
            // ... order logic ...
            return nil
        })
    })

    return order, err
}

```

Custom retry condition

```

cfg := advanced.RetryConfig{
    MaxAttempts: 5,
    RetryIf: func(err error) bool {
        // Custom logic
        return errors.Is(err, sql.ErrConnDone) ||
            strings.Contains(err.Error(), "timeout")
    },
}

```

Best practices

- ✓ **Always retry transient DB errors** (deadlocks, timeouts)
- ✓ **Use exponential backoff** with jitter
- ✓ **Cap max attempts** (5-10 is reasonable)
- ✓ **Cap max delay** (5-30 seconds)
- ✗ **Never retry** validation errors
- ✗ **Never retry** 4xx HTTP errors

25. Batch Processing

Parallel batch processing

Process items in parallel workers:

```
users := []User{ /* 100,000 users */ }

result := advanced.ProcessBatch(ctx, users,
    advanced.DefaultBatchConfig(), // 4 workers, batch size 500
    func(batch []User) error {
        return gormz.New[User]().CreateMany(batch)
    })

fmt.Println("Total: ", result.Total)
fmt.Println("Success:", result.Success)
fmt.Println("Failed: ", result.Failed)
fmt.Println("Errors: ", len(result.Errors))
```

Custom config

```
cfg := advanced.BatchConfig{
    BatchSize: 500,
    Workers:   8,
    StopOnError: true, // stop on first error
}

result := advanced.ProcessBatch(ctx, users, cfg, func(batch []User) error {
    return processBatch(batch)
})
```

Streaming from DB

Process large query results in chunks:

```
err := advanced.Stream[User](ctx, 1000, func(batch []User) error {
    for _, u := range batch {
        if err := sendWelcomeEmail(u); err != nil {
```

```
        return err
    }
}
return nil
}))
```

Parallel independent tasks

```
p := advanced.Parallel().WithWorkers(4)
p.Add(func() error { return task1(ctx) })
p.Add(func() error { return task2(ctx) })
p.Add(func() error { return task3(ctx) })

err := p.Run(ctx)
```

Real-world: nightly cleanup

```
func NightlyCleanup(ctx context.Context) error {
    // 1. Delete old sessions in batches
    _, err := advanced.BulkDeleteWhere[Session](ctx,
        "expires_at < ?", time.Now().AddDate(0, 0, -30))
    if err != nil {
        return err
    }

    // 2. Archive old orders
    err = advanced.Stream[Order](ctx, 500, func(batch []Order) {
        for _, order := range batch {
            if err := archiveOrder(order); err != nil {
                return err
            }
        }
        return nil
    })
    if err != nil {
        return err
    }
}
```

```

    }

    // 3. Recalculate stats in parallel
    p := advanced.Parallel().WithWorkers(4)
    p.Add(func() error { return recalcUserStats(ctx) })
    p.Add(func() error { return recalcProductStats(ctx) })
    p.Add(func() error { return recalcCategoryStats(ctx) })

    return p.Run(ctx)
}

```

API reference

```

type BatchConfig struct {
    BatchSize    int    // records per batch
    Workers      int    // parallel workers
    StopOnError  bool   // stop on first error
}

func DefaultBatchConfig() BatchConfig {
    return BatchConfig{BatchSize: 1000, Workers: 4}
}

type BatchResult struct {
    Total      int
    Success    int
    Failed     int
    Errors     []error
}

advanced.ProcessBatch[T](ctx, items, cfg, fn) *BatchResult
advanced.Stream[T](ctx, batchSize, fn) error
advanced.Parallel() *ParallelQuery

```

Best practices

- ✓ **Use 4 workers** for I/O-bound tasks
 - ✓ **Use 1-2 workers** for CPU-bound tasks
 - ✓ **Batch size 500-1000** for DB writes
 - ✓ **Handle errors** — don't ignore them
 - ⚠ **SQLite doesn't like parallel writes** — use 1 worker
 - ✗ **Don't** create goroutines inside `fn`
-

26. Error Handling

Typed errors

gormz provides rich error types:

```
var (
    ErrNotFound          = gorm.ErrRecordNotFound
    ErrNotInitialized    = errors.New("gormz: DB not initialized")
    ErrNilDB              = errors.New("gormz: nil DB")
    ErrInvalidField       = errors.New("gormz: invalid field")
    ErrInvalidQuery       = errors.New("gormz: invalid query")
    ErrDangerousOperation = errors.New("gormz: dangerous operation")
    ErrAlreadyRegistered  = errors.New("gormz: model already registered")
    ErrNotFoundInRegistry = errors.New("gormz: model not found in registry")
)
```

Specific error types

NotFoundError:

```
type NotFoundError struct {
    Model string // "User"
    ID     any     // 42
}
```

```
err := gormz.NewNotFoundError("User", 42)
// gormz: User with id 42 not found
```

ValidationError:

```
type ValidationError struct {
    Field  string // "email"
    Reason string // "invalid format"
}

err := gormz.NewValidationError("email", "invalid format")
// gormz: invalid field "email": invalid format
```

DangerousOperationError:

```
type DangerousOperationError struct {
    Operation string // "DeleteMany"
    Reason     string  // "requires conditions"
}
```

Error checks

```
if gormz.IsNotFound(err) { ... }
if gormz.IsValidation(err) { ... }
if gormz.IsDangerous(err) { ... }
if gormz.IsAlreadyRegistered(err) { ... }
if gormz.IsNotFoundInRegistry(err) { ... }
```

Extract details

```
if ne, ok := gormz.AsNotFound(err); ok {
    fmt.Println("Model:", ne.Model)
    fmt.Println("ID:", ne.ID)
}
```

```

if ve, ok := gormz.AsValidation(err); ok {
    fmt.Println("Field:", ve.Field)
    fmt.Println("Reason:", ve.Reason)
}

if de, ok := gormz.AsDangerous(err); ok {
    fmt.Println("Operation:", de.Operation)
    fmt.Println("Reason:", de.Reason)
}

```

Works with errors.Is / errors.As

```

// errors.Is
if errors.Is(err, gormz.ErrNotFound) {
    // handle not found
}

// errors.As
var nfErr *gormz.NotFoundError
if errors.As(err, &nfErr) {
    log.Printf("Not found: %s#%v", nfErr.Model, nfErr.ID)
}

```

HTTP error handler

```

func HandleError(c *fiber.Ctx, err error) error {
    if err == nil {
        return nil
    }

    // Not found
    if ne, ok := gormz.AsNotFound(err); ok {
        return c.Status(404).JSON(fiber.Map{
            "error": "not found",
            "model": ne.Model,
            "id":    ne.ID,
        })
    }
}

```

```

        })
    }

    // Validation
    if ve, ok := gormz.AsValidation(err); ok {
        return c.Status(400).JSON(fiber.Map{
            "error": "validation failed",
            "field": ve.Field,
            "reason": ve.Reason,
        })
    }

    // Dangerous operation
    if de, ok := gormz.AsDangerous(err); ok {
        return c.Status(400).JSON(fiber.Map{
            "error": "dangerous operation",
            "operation": de.Operation,
        })
    }

    // Default 500
    return c.Status(500).JSON(fiber.Map{"error": "internal server error"})
}

```

Wrapping errors

```

user, err := gormz.New[User]().Get(id)
if err != nil {
    return fmt.Errorf("get user %d: %w", id, err)
}

```

Best practices

- ✓ **Always check errors**
- ✓ **Wrap with context** (`fmt.Errorf("...: %w", err)`)
- ✓ **Use typed errors** for domain errors

- ✓ **Log internal errors** (never return raw)
 - ✗ **Never ignore errors** silently
 - ✗ **Never panic()** in library code
-

27. Security

SQL injection protection

gormz validates **every field name** before using it in SQL.

Safe:

```
q.Filter("name", "Ali")           // ✓
q.Filter("email__icontains", "@x") // ✓
```

Panics:

```
q.Filter("name; DROP TABLE users", "x") // ✗ panic
q.Filter("name' OR '1'='1", "x")         // ✗ panic
q.Filter("1=1 --", "x")                   // ✗ panic
```

For user input:

```
// ✓ Safe
q, err := gormz.New[User]().TryFilter(userInput, value)
if err != nil {
    return c.Status(400).JSON(fiber.Map{"error": "invalid"})
}
```

Dangerous operation guards

gormz **refuses** to run destructive operations without conditions:

```
// ✖ Error
_, err := gormz.New[User]().DeleteMany()
// DangerousOperationError

// ✖ Error
_, err := gormz.New[User]().UpdateMany(map[string]any{"x": 1})
// DangerousOperationError

// ✖ Error
_, err := gormz.New[User]().RestoreAll()
// DangerousOperationError

// ✔ Works — has condition
_, err := gormz.New[User]().Filter("active", false).DeleteMany()
```

Raw SQL — user responsibility

```
// ⚠ gormz doesn't validate raw SQL
q.Where("age > ? AND status = ?", 18, "active")

// ✔ Always parameterize
q.Where("email = ?", userEmail)

// ✖ Never concatenate
q.Where(fmt.Sprintf("email = '%s'", userEmail)) // SQL injection
```

Field validation

```
// Valid fields
"name", "email", "user_id", "users.id", "t1.name"
```

Invalid fields (rejected):

- "" — empty
- "name; DROP TABLE" — SQL injection
- "select" — SQL keyword

- "1field" — starts with number
- "user name" — spaces

Best practices

- ✓ **Always use `Try*` for external input**
 - ✓ **Validate user input** before querying
 - ✓ **Use parameterized queries** in `Where()`
 - ✓ **Sanitize output** before displaying
 - ✓ **Hash passwords** (never store plain)
 - ✓ **Use HTTPS** in production
 - ✓ **Enable CSRF protection** (framework-level)
 - ✓ **Log security events** (failed logins, etc.)
-

28. Performance Tuning

Connection pool

Configure based on workload:

```
cfg := gormz.DefaultConfig()

// For high-traffic API
cfg.MaxOpenConns = 100
cfg.MaxIdleConns = 25
cfg.ConnMaxLifetime = 2 * time.Hour
cfg.ConnMaxIdleTime = 30 * time.Minute

// For low-traffic background jobs
cfg.MaxOpenConns = 10
cfg.MaxIdleConns = 2
cfg.ConnMaxLifetime = 1 * time.Hour

gormz.Configure(cfg)
```

Rule of thumb:

- `MaxOpenConns` = (number of CPUs × 2) to (number of CPUs × 4)
- `MaxIdleConns` = `MaxOpenConns` / 4
- `ConnMaxLifetime` = 1–2 hours (avoid stale connections)

Index strategy

Index frequently filtered columns:

```
type User struct {
    ID      uint
    Email    string `gorm:"uniqueIndex;size:255"` // unique
    Status   string `gorm:"index"`               // frequently filtered
    Age      int    `gorm:"index"`               // for sorting
}
```

Composite index for multi-column filters:

```
type Order struct {
    TenantID uint `gorm:"index:idx_tenant_status,priority:1"`
    Status    string `gorm:"index:idx_tenant_status,priority:2"`
}
// Optimizes: WHERE tenant_id = ? AND status = ?
```

Query optimization

Select only needed fields:

```
// ✗ All columns
users, _ := gormz.New[User]().All()

// ✓ Just what you need
users, _ := gormz.New[User]().
    Select("id", "name", "email").
    All()
```

Use pagination:

```
// ✗ Loads entire table
users, _ := gormz.New[User]().All()

// ✓ Just one page
page, _ := gormz.New[User]().Paginate(1, 20)
```

Avoid N+1:

```
// ✗ N+1
users, _ := gormz.New[User]().All()
for _, u := range users {
    orders, _ := gormz.New[Order]().Filter("user_id", u.ID)
}

// ✓ 2 queries
users, _ := gormz.New[User]().Preload("Orders").All()
```

Caching

Cache expensive queries:

```
var userCount int64
cacheKey := "stats:user_count"

if err := cache.Get(cacheKey, &userCount); err != nil {
    // Cache miss - query DB
    userCount, _ = gormz.New[User]().Count()
    cache.Set(cacheKey, userCount, 5*time.Minute)
}
```

Slow query logging

```
cfg := gormz.DefaultConfig()
```

```
cfg.LogLevel = gormz.LogWarn
cfg.SlowQuery = 100 * time.Millisecond // log queries >
gormz.Configure(cfg)
```

EXPLAIN queries

```
sql, args := q.ToSQL()
fmt.Println(sql)

// In DB shell
// EXPLAIN ANALYZE <sql>
```

Benchmarks

Reference (SQLite, Intel i7, 16GB RAM):

Operation	Time
Create (single)	~100µs
Filter	~26µs
Paginate	~68µs
Preload	~150µs
Bulk Insert (1000)	~1.5ms

Performance checklist

- ✓ Tune connection pool for your workload
- ✓ Add indexes on filtered/sorted columns
- ✓ Use `Select` for needed columns only
- ✓ Always paginate large results
- ✓ Use `Preload` (avoid N+1)
- ✓ Use `CreateInBatches` for large inserts
- ✓ Add `Context` with timeout
- ✓ Cache expensive aggregates

- ✓ Use `EXPLAIN` for slow queries
 - ✓ Log slow queries
-

29. Migration from GORM

Why migrate?

- Type safety
- Immutability
- Cleaner code
- Better pagination
- Advanced features (CTEs, Windows)
- Typed errors
- Safety guards

Gradual migration strategy

You can use **both** in the same project:

```
// Old code — keep using GORM
db.Where("age > ?", 18).Find(&users)

// New code — use gormz
gormz.New[User]().Filter("age__gt", 18).All()
```

Steps:

1. **Setup gormz** alongside GORM:

```
db, _ := gorm.Open(...)
gormz.SetDB(db) // shares the same connection
```

2. **Migrate one feature at a time:**

- Start with new endpoints
- Migrate existing code gradually
- Test thoroughly

3. Keep GORM hooks — they still work:

```
func (u *User) BeforeCreate(tx *gorm.DB) error {
    u.Email = strings.ToLower(u.Email)
    return nil
}
```

Conversion table

GORM	gormz
<code>db.Where("age > ?", 18).Find(&u)</code>	<code>gormz.New[U]().Filter("age__g</code>
<code>db.First(&user, 1)</code>	<code>gormz.New[U]().Ge</code>
<code>db.First(&user, "email = ?", "x")</code>	<code>gormz.New[U]().Fi</code> <code>"x")</code>
<code>db.Create(&user)</code>	<code>gormz.New[U]().Cr</code>
<code>db.Save(&user)</code>	<code>gormz.New[U]().Sa</code>
<code>db.Delete(&user, 1)</code>	<code>gormz.New[U]().De</code>
<code>db.Model(&u).Update("x", "y")</code>	<code>gormz.New[U]().Up</code> <code>"y")</code>
<code>db.Model(&u).Updates(map)</code>	<code>gormz.New[U]</code> <code>() .Filter(...).Up</code>
<code>db.Model(&u).Count(&c)</code>	<code>gormz.New[U]().Co</code>
<code>db.Model(&u).Preload("Orders").Find(&u)</code>	<code>gormz.New[U]</code> <code>() .Preload("Order</code>
<code>db.Transaction(fn)</code>	<code>gormz.Transaction</code>
<code>db.Raw(...).Scan(&dest)</code>	<code>gormz.New[U]().Sc</code>

Before / After comparison

Before (GORM) — 20 lines:

```
func GetActiveUsers(db *gorm.DB, search string, minAge int) ([]User, error) {
    var users []User
    query := db.Model(&User{}).Where("active = ?", true)

    if search != "" {
        query = query.Where("name LIKE ?", "%"+search+"%")
    }
    if minAge > 0 {
        query = query.Where("age >= ?", minAge)
    }

    err := query.
        Order("created_at DESC").
        Limit(20).
        Preload("Orders").
        Find(&users).Error

    return users, err
}
```

After (gormz) — 12 lines:

```
func GetActiveUsers(ctx context.Context, search string, minAge int) ([]User, error) {
    q := gormz.New[User]().Filter("active", true)

    q = q.FilterIf(search != "", "name__contains", search)
    q = q.FilterIf(minAge > 0, "age__gte", minAge)

    return q.
        OrderBy("-created_at").
        Limit(20).
        Preload("Orders").
        WithContext(ctx).
        All()
}
```

```
}
```

Improvements:

- ✓ Type-safe
- ✓ Context-aware
- ✓ Shorter
- ✓ Immutable
- ✓ SQL injection safe

Common migration patterns

Pattern 1 — Dynamic filters:

```
// GORM
query := db.Model(&User{})
if name != "" { query = query.Where("name = ?", name) }
if age > 0 { query = query.Where("age >= ?", age) }

// gormz
q := gormz.New[User]()
q = q.FilterIf(name != "", "name", name)
q = q.FilterIf(age > 0, "age__gte", age)
```

Pattern 2 — Complex OR:

```
// GORM
db.Where("status = ? OR status = ?", "active", "pending")

// gormz
q := gormz.Or(
    gormz.Eq("status", "active"),
    gormz.Eq("status", "pending"),
)
gormz.New[User]() .Q(q) .All()
```

Pattern 3 — Pagination:

```
// GORM - manual
var total int64
db.Model(&User{}).Where("active = ?", true).Count(&total)
db.Where("active = ?", true).
    Limit(20).Offset(0).
    Find(&users)

// gormz - built-in
page, _ := gormz.New[User]().
    Filter("active", true).
    Paginate(1, 20)
// page.Items, page.Total, page.Page, ...
```

Pattern 4 — Subqueries:

```
// GORM
db.Where("id IN (?)",
    db.Model(&Order{}).Select("user_id").Where("status = ")
).Find(&users)

// gormz
subq := advanced.SubFrom[Order](
    gormz.New[Order]().Select("user_id").Filter("status",
        "user_id",
    )
gormz.New[User]().Q(advanced.In("id", subq)).All()
```

Testing migration

```
func TestMigrationToGormz(t *testing.T) {
    // Old way
    var oldUsers []User
    db.Where("age > ?", 18).Find(&oldUsers)

    // New way
    newUsers, _ := gormz.New[User]().Filter("age__gt", 18)
```

```
// Same result
assert.Equal(t, len(oldUsers), len(newUsers))
}
```

Advantages checklist

- ✓ **Type-safe** — no string concatenation
 - ✓ **Immutable** — no shared state
 - ✓ **Thread-safe** — safe concurrent use
 - ✓ **SQL injection proof** — validated fields
 - ✓ **Shorter code** — less boilerplate
 - ✓ **Built-in pagination** — no manual count
 - ✓ **Advanced lookups** — `__gt`, `__in`, ...
 - ✓ **Typed errors** — better error handling
 - ✓ **Advanced features** — CTEs, Windows, Unions
 - ✓ **Same GORM** — no new dependencies
-

Part 6 — Complete Project

30. Building a Blog API — Full Walkthrough

Let's build a **production-ready Blog API** with gormz from scratch.

Project structure

```
blog-api/
├── cmd/
│   └── server/
│       └── main.go
└── internal/
```

```
|   └─ models/
|       └─ base.go
|       └─ user.go
|       └─ post.go
|       └─ comment.go
|       └─ tag.go
|   └─ dto/
|       └─ user_dto.go
|       └─ post_dto.go
|       └─ comment_dto.go
|   └─ services/
|       └─ auth_service.go
|       └─ user_service.go
|       └─ post_service.go
|       └─ comment_service.go
|   └─ handlers/
|       └─ auth_handler.go
|       └─ post_handler.go
|       └─ comment_handler.go
|   └─ middleware/
|       └─ auth.go
|   └─ routes/
|       └─ routes.go
└─ config/
    └─ config.go
└─ go.mod
└─ README.md
```

Step 1 — Setup project

```
mkdir blog-api && cd blog-api
go mod init github.com/yourname/blog-api

go get github.com/light-tech-dev/gormz
go get github.com/gofiber/fiber/v2
go get github.com/golang-jwt/jwt/v5
go get golang.org/x/crypto
go get gorm.io/driver/sqlite
```

```
go get gorm.io/gorm
```

Step 2 — Config

config/config.go:

```
package config

import (
    "os"

    "github.com/joho/godotenv"
)

type Config struct {
    Port      string
    DBPath    string
    JWTSecret string
}

func Load() *Config {
    _ = godotenv.Load()

    return &Config{
        Port:      getEnv("PORT", "8080"),
        DBPath:    getEnv("DB_PATH", "blog.db"),
        JWTSecret: getEnv("JWT_SECRET", "dev-secret-chang
    }
}

func getEnv(key, def string) string {
    if v := os.Getenv(key); v != "" {
        return v
    }
    return def
}
```

Step 3 — Models

internal/models/base.go:

```
package models

import "time"

type BaseModel struct {
    ID          uint          `gorm:"primaryKey" json:"id"`
    CreatedAt   time.Time   `json:"created_at"`
    UpdatedAt   time.Time   `json:"updated_at"`
}
```

internal/models/user.go:

```
package models

import (
    "time"

    "gorm.io/gorm"
)

type User struct {
    BaseModel
    Username string    `gorm:"uniqueIndex;size:50;not null" json:"username"`
    Email     string    `gorm:"uniqueIndex;size:255;not null" json:"email"`
    Password  string    `gorm:"not null" json:"password"`
    FullName  string    `gorm:"size:255" json:"full_name"`
    Bio       string    `gorm:"type:text" json:"bio"`
    IsActive  bool      `gorm:"default:true" json:"is_active"`
    LastLogin *time.Time `json:"last_login,omitempty"`

    Posts []Post    `gorm:"foreignKey:AuthorID" json:"posts"`
    Comments []Comment `gorm:"foreignKey:AuthorID" json:"comments"`
}

func (User) TableName() string { return "users" }
```

```
func (u *User) BeforeCreate(tx *gorm.DB) error {
    // normalize
    return nil
}
```

internal/models/post.go:

```
package models

import "time"

type PostStatus string

const (
    PostStatusDraft      PostStatus = "draft"
    PostStatusPublished PostStatus = "published"
    PostStatusArchived   PostStatus = "archived"
)

type Post struct {
    BaseModel
    Title      string      `gorm:"size:255;not null;index"`
    Slug       string      `gorm:"uniqueIndex;size:255;not null;index"`
    Content    string      `gorm:"type:text;not null" json:"content"`
    Excerpt    string      `gorm:"size:500" json:"excerpt"`
    Status     PostStatus  `gorm:"size:20;default:'draft'"`
    PublishedAt *time.Time `gorm:"index" json:"published_at"`
    ViewCount  int         `gorm:"default:0" json:"view_count"`

    AuthorID uint `gorm:"index;not null" json:"author_id"`
    Author   *User `gorm:"foreignKey:AuthorID" json:"author"`

    Comments []Comment `gorm:"foreignKey:PostID" json:"comments"`
    Tags     []Tag      `gorm:"many2many:post_tags;" json:"tags"`
}

func (Post) TableName() string { return "posts" }
```

internal/models/comment.go:

```
package models

type Comment struct {
    BaseModel
    Content string `gorm:"type:text;not null" json:"content"`
    IsActive bool `gorm:"default:true" json:"is_active"`

    PostID uint `gorm:"index;not null" json:"post_id"`
    Post *Post `gorm:"foreignKey:PostID" json:"post"`
    AuthorID uint `gorm:"index;not null" json:"author_id"`
    Author *User `gorm:"foreignKey:AuthorID" json:"author"`

    ParentID *uint `gorm:"index" json:"parent_id,omitempty"`
    Parent *Comment `gorm:"foreignKey:ParentID" json:"parent"`
    Replies []Comment `gorm:"foreignKey:ParentID" json:"replies"`
}

func (Comment) TableName() string { return "comments" }
```

internal/models/tag.go:

```
package models

type Tag struct {
    BaseModel
    Name string `gorm:"uniqueIndex;size:50;not null" json:"name"`
    Slug string `gorm:"uniqueIndex;size:50;not null" json:"slug"`

    Posts []Post `gorm:"many2many:post_tags;" json:"posts"`
}

func (Tag) TableName() string { return "tags" }
```

Step 4 — DTOs

internal/dto/user_dto.go:

```
package dto

import "time"

type RegisterDTO struct {
    Username string `json:"username"`
    Email     string `json:"email"`
    Password  string `json:"password"`
    FullName  string `json:"full_name"`
}

type LoginDTO struct {
    Username string `json:"username"`
    Password string `json:"password"`
}

type UserResponse struct {
    ID          uint      `json:"id"`
    Username    string    `json:"username"`
    Email       string    `json:"email"`
    FullName    string    `json:"full_name"`
    Bio         string    `json:"bio"`
    CreatedAt   time.Time `json:"created_at"`
}
```

internal/dto/post_dto.go:

```
package dto

import "time"

type CreatePostDTO struct {
    Title       string    `json:"title"`
    Content     string    `json:"content"`
    Excerpt     string    `json:"excerpt"`
    Status      string    `json:"status"`
    TagSlugs    []string  `json:"tag_slugs"`
}
```

```

}

type UpdatePostDTO struct {
    Title      string `json:"title"`
    Content    string `json:"content"`
    Excerpt    string `json:"excerpt"`
    Status     string `json:"status"`
}

type PostResponse struct {
    ID          uint      `json:"id"`
    Title       string    `json:"title"`
    Slug        string    `json:"slug"`
    Content     string    `json:"content"`
    Excerpt     string    `json:"excerpt"`
    Status      string    `json:"status"`
    ViewCount   int       `json:"view_count"`
    Author      *UserResponse `json:"author,omitempty"`
    Tags        []string  `json:"tags"`
    PublishedAt *time.Time `json:"published_at,omitempty"`
    CreatedAt   time.Time `json:"created_at"`
    UpdatedAt   time.Time `json:"updated_at"`
}

```

Step 5 — Services

internal/services/post_service.go:

```

package services

import (
    "context"
    "errors"
    "fmt"
    "strings"
    "time"

    "github.com/light-tech-dev/gormz"

```

```

    "github.com/light-tech-dev/gormz/advanced"

    "github.com/yourname/blog-api/internal/dto"
    "github.com/yourname/blog-api/internal/models"
)

type PostService struct{}

func NewPostService() *PostService {
    return &PostService{}
}

// List - paginated with filters
func (s *PostService) List(ctx context.Context, page, perPage int) ([]*dto.Post, error) {
    q := gormz.New[models.Post]().
        WithContext(ctx).
        Filter("status", string(models.PostStatusPublished)).
        Preload("Author").
        Preload("Tags")

    if search != "" {
        q = q.Q(gormz.Or(
            gormz.Contains("title", search),
            gormz.Contains("content", search),
        ))
    }

    return q.OrderBy("-published_at").Paginate(page, perPage)
}

// GetBySlug - with full relations
func (s *PostService) GetBySlug(ctx context.Context, slug string) (*dto.Post, error) {
    post, err := gormz.New[models.Post]().
        WithContext(ctx).
        Filter("slug", slug).
        Filter("status", string(models.PostStatusPublished)).
        Preload("Author").
        Preload("Tags").
        Preload("Comments").
        Preload("Comments.Author").

```

```

        First()

    if err != nil {
        if gormz.IsNotFound(err) {
            return nil, errors.New("post not found")
        }
        return nil, err
    }

    // Increment view count (fire-and-forget)
    go func() {
        gormz.New[models.Post]().
            Where("id = ?", post.ID).
            UpdateColumn(post.ID, "view_count", post.ViewCount)
    }()

    return post, nil
}

// Create - with tags
func (s *PostService) Create(ctx context.Context, authorID int64, req *models.PostReq) (*models.Post, error) {
    var post *models.Post

    err := advanced.WithTransaction(ctx, advanced.DefaultTx) func(tx *advanced.Tx) error {
        // Generate slug
        slug := generateSlug(req.Title)

        // Set status
        status := models.PostStatusDraft
        if req.Status == "published" {
            status = models.PostStatusPublished
        }

        // Create post
        post = &models.Post{
            Title:    req.Title,
            Slug:     slug,
            Content:  req.Content,
            Excerpt:  req.Excerpt,
            AuthorID: authorID,
            Status:  status,
        }

        return s.CreatePost(tx, post)
    }

    if err != nil {
        return nil, err
    }

    return post, nil
}

```

```

        Status:    status,
        AuthorID: authorID,
    }

    if status == models.PostStatusPublished {
        now := time.Now()
        post.PublishedAt = &now
    }

    if err := advanced.Query[models.Post](tx).Create(&post); err != nil {
        return err
    }

    // Attach tags
    if len(req.TagSlugs) > 0 {
        var tags []models.Tag
        if err := tx.DB().Where("slug IN ?", req.TagSlugs).Find(&tags); err != nil {
            return err
        }
        if err := tx.DB().Model(post).Association("Tags").Create(&tags); err != nil {
            return err
        }
    }

    return nil
})

if err != nil {
    return nil, err
}

return post, nil
}

// Helper
func generateSlug(title string) string {
    slug := strings.ToLower(title)
    slug = strings.ReplaceAll(slug, " ", "-")
    slug = strings.ReplaceAll(slug, "_", "-")
}

```

```

    // Remove non-alphanumeric
    var b strings.Builder
    for _, r := range slug {
        if (r >= 'a' && r <= 'z') || (r >= '0' && r <= '9') {
            b.WriteRune(r)
        }
    }

    return fmt.Sprintf("%s-%d", b.String(), time.Now().Unix())
}

```

Step 6 — Handlers

internal/handlers/post_handler.go:

```

package handlers

import (
    "strconv"

    "github.com/gofiber/fiber/v2"

    "github.com/yourname/blog-api/internal/dto"
    "github.com/yourname/blog-api/internal/services"
)

type PostHandler struct {
    service *services.PostService
}

func NewPostHandler() *PostHandler {
    return &PostHandler{service: services.NewPostService()}
}

func (h *PostHandler) List(c *fiber.Ctx) error {
    page, _ := strconv.Atoi(c.Query("page", "1"))
    perPage, _ := strconv.Atoi(c.Query("per_page", "20"))
    search := c.Query("q")

```

```

        result, err := h.service.List(c.Context(), page, perP
        if err != nil {
            return c.Status(500).JSON(fiber.Map{"error": err.
        }

        return c.JSON(result)
    }

func (h *PostHandler) Get(c *fiber.Ctx) error {
    slug := c.Params("slug")

    post, err := h.service.GetBySlug(c.Context(), slug)
    if err != nil {
        return c.Status(404).JSON(fiber.Map{"error": err.
    }

    return c.JSON(post)
}

func (h *PostHandler) Create(c *fiber.Ctx) error {
    userID := c.Locals("user_id").(uint)

    req := new(dto.CreatePostDTO)
    if err := c.BodyParser(req); err != nil {
        return c.Status(400).JSON(fiber.Map{"error": "inv
    }

    post, err := h.service.Create(c.Context(), userID, re
    if err != nil {
        return c.Status(400).JSON(fiber.Map{"error": err.
    }

    return c.Status(201).JSON(post)
}

```

Step 7 — Routes

internal/routes/routes.go:

```
package routes

import (
    "github.com/gofiber/fiber/v2"

    "github.com/yourname/blog-api/internal/handlers"
    "github.com/yourname/blog-api/internal/middleware"
)

func Setup(app *fiber.App, jwtSecret string) {
    api := app.Group("/api/v1")

    // Health
    api.Get("/health", func(c *fiber.Ctx) error {
        return c.JSON(fiber.Map{"status": "ok"})
    })

    // Public
    postHandler := handlers.NewPostHandler()
    api.Get("/posts", postHandler.List)
    api.Get("/posts/:slug", postHandler.Get)

    // Protected
    protected := api.Group("", middleware.AuthRequired(jw
    protected.Post("/posts", postHandler.Create)
}
```

Step 8 — Main

cmd/server/main.go:

```
package main

import (
    "log"
```

```

"github.com/gofiber/fiber/v2"
"github.com/gofiber/fiber/v2/middleware/cors"
"github.com/gofiber/fiber/v2/middleware/logger"
"github.com/gofiber/fiber/v2/middleware/recover"

"github.com/light-tech-dev/gormz"
"gorm.io/driver/sqlite"
"gorm.io/gorm"

"github.com/yourname/blog-api/config"
"github.com/yourname/blog-api/internal/models"
"github.com/yourname/blog-api/internal/routes"
)

func main() {
    cfg := config.Load()

    // DB
    db, err := gorm.Open(sqlite.Open(cfg.DBPath), &gorm.C
    if err != nil {
        log.Fatal(err)
    }
    gormz.SetDB(db)

    // Migrate
    if err := gormz.MigrateAll(
        &models.User{},
        &models.Post{},
        &models.Comment{},
        &models.Tag{},
    ); err != nil {
        log.Fatal(err)
    }
    log.Println("✓ Database migrated")

    // Fiber
    app := fiber.New(fiber.Config{AppName: "blog-api"})
    app.Use(recover.New())
    app.Use(logger.New())
    app.Use(cors.New())

```

```
// Routes
routes.Setup(app, cfg.JWTSecret)

// Listen
log.Printf("🦉 Server on http://localhost:%s", cfg.Port)
log.Fatal(app.Listen(":" + cfg.Port))
}
```

Step 9 — Run it

```
go run cmd/server/main.go
```

Step 10 — Test the API

```
# Health
curl http://localhost:8080/api/v1/health

# List posts (public)
curl http://localhost:8080/api/v1/posts

# Get post by slug
curl http://localhost:8080/api/v1/posts/my-first-post

# Create post (requires auth)
TOKEN="..."
curl -X POST http://localhost:8080/api/v1/posts \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "title": "My First Post",
    "content": "Hello world!",
    "status": "published",
    "tag_slugs": ["golang", "tutorial"]
  }'
```

What we learned

This project demonstrates:

- ✓ **Models** — User, Post, Comment, Tag with relations
 - ✓ **DTOs** — Request/response objects
 - ✓ **Services** — Business logic
 - ✓ **Handlers** — HTTP layer
 - ✓ **Routes** — URL mapping
 - ✓ **Transactions** — Multi-step operations
 - ✓ **Preloading** — Relations without N+1
 - ✓ **Pagination** — Built-in
 - ✓ **Advanced lookups** — Search across fields
 - ✓ **Context** — Full propagation
 - ✓ **Middleware** — Auth
-

Appendices

Appendix A — Full API Reference

Setup

```
gormz.SetDB(db)           // bind connection
gormz.DB() *gorm.DB       // current connection
gormz.IsReady() bool      // is DB ready?
gormz.ResetDB()           // reset (for tests)
gormz.Configure(cfg) error // set pool config
gormz.Ping() error        // check connection
gormz.Close() error       // close connection
```

Migrate

```
gormz.Migrate[T]() error
gormz.MustMigrate[T]()
gormz.MigrateAll(models...) error
gormz.MustMigrateAll(models...)
gormz.DropTable[T]() error
gormz.HasTable[T]() bool
```

QuerySet — Construction

```
gormz.New[T]() // *QuerySet[T]
gormz.NewWith[T](instance) // on Instance
gormz.FromContext[T](ctx) // from context
gormz.MustFromContext[T](ctx) // from context (panic)
```

QuerySet — Filters

```
.Filter(field, value)
.TryFilter(field, value)
.FilterIf(cond, field, value)
.Exclude(field, value)
.TryExclude(field, value)
.Where(sql, args...)
.Q(builder *Q)
```

QuerySet — Ordering

```
.OrderBy(fields...)
.TryOrderBy(fields...)
```

QuerySet — Limits

```
.Limit(n)
.Offset(n)
.Page(page, perPage)
```

QuerySet — Selection

```
.Select(fields...)
.TrySelect(fields...)
.SelectRaw(exprs...)
.Omit(fields...)
.TryOmit(fields...)
```

QuerySet — Relations

```
.Preload(relations...)
```

QuerySet — Soft Delete

```
.WithDeleted()
.OnlyDeleted()
```

QuerySet — Grouping

```
.GroupBy(fields...)
.TryGroupBy(fields...)
.Having(sql, args...)
.Distinct()
.DistinctOn(fields...)
```

QuerySet — Joins

```
.Join(type, table, on)
.InnerJoin(table, on)
.LeftJoin(table, on)
.RightJoin(table, on)
.CrossJoin(table)
```

QuerySet — Window

```
.Window(exprs...)
```

QuerySet — Context

```
.WithContext(ctx)
.Context() context.Context
```

QuerySet — Reads

```
.All() // ([]T, error)
.First() // (*T, error)
.FirstOrNil() // (*T, error)
.Last() // (*T, error)
.Take() // (*T, error)
.Get(id) // (*T, error)
.GetOrNil(id) // (*T, error)
.Find(field, val) // (*T, error)
.FindOrNil(field, val)
.Count() // (int64, error)
.Exists() // (bool, error)
.Pluck(field, dest) // error
.ScanInto(dest) // error
.Sum(field) // (float64, error)
.Avg(field) // (float64, error)
.Min(field) // (float64, error)
.Max(field) // (float64, error)
```

```
.Paginate(p, pp) // (*PaginatedResult[T], error)
```

QuerySet — Writes

```
.Create(item) // error
.CreateMany(items) // error
.CreateInBatches(items, size) // error
.Save(item) // error
.Update(id, field, value) // error
.UpdateColumn(id, field, value) // error
.UpdateMany(values) // (int64, error)
.Delete(id) // error
.DeleteMany() // (int64, error)
.HardDelete(id) // error
.Restore(id) // error
.RestoreAll() // (int64, error)
```

QuerySet — Debug

```
.ToSQL() (string, []any)
.String() string
.DryRun() *gorm.DB
```

Q Builder

```
gormz.Qb() *Q
gormz.Or(clauses...) *Q
gormz.And(clauses...) *Q

gormz.Eq(field, value)
gormz.Ne(field, value)
gormz.Gt(field, value)
gormz.Gte(field, value)
gormz.Lt(field, value)
```

```
gormz.Lte(field, value)
gormz.Contains(field, value)
gormz.StartsWith(field, value)
gormz.EndsWith(field, value)
gormz.In(field, values)
gormz.IsNull(field)
gormz.NotNull(field)
gormz.Raw(sql, args...)
gormz.Not(clause)
```

Registry

```
gormz.Register[T](name) *QuerySet[T]
gormz.TryRegister[T](name) (*QuerySet[T], error)
gormz.Lookup[T](name) (*QuerySet[T], bool)
gormz.MustLookup[T](name) *QuerySet[T]
gormz.Has(name) bool
gormz.Unregister(name)
gormz.RegisteredNames() []string
gormz.RegisteredCount() int
gormz.ClearRegistry()
gormz.RegisterBatch(entries) error
gormz.UnregisterBatch(names...) int
gormz.HasType[T]() bool
gormz.LookupByType[T]() (*QuerySet[T], bool)
gormz.MustLookupByType[T]() *QuerySet[T]
gormz.NamesByType[T]() []string
gormz.TakeSnapshot() Snapshot
```

Errors

```
gormz.IsNotFound(err) bool
gormz.IsValidation(err) bool
gormz.IsDangerous(err) bool
gormz.IsAlreadyRegistered(err) bool
gormz.IsNotFoundInRegistry(err) bool
```

```
gormz.AsNotFound(err) (*NotFoundError, bool)
gormz.AsValidation(err) (*ValidationError, bool)
gormz.AsDangerous(err) (*DangerousOperationError, bool)

gormz.NewNotFoundError(model, id)
gormz.NewValidationError(field, reason)
gormz.NewDangerousError(operation, reason)
```

Instance

```
gormz.NewInstance(db) *Instance
instance.DB() *gorm.DB
instance.Ping() error
instance.Close() error
instance.Configure(cfg) error
instance.Transaction(ctx, fn) error
instance.Migrate(models...) error

gormz.GlobalInstance() *Instance
gormz.NewWith[T](instance) *QuerySet[T]
gormz.QueryOn[T](instance) *QuerySet[T]
gormz.QueryOnWithContext[T](i, ctx) *QuerySet[T]
```

Context

```
gormz.WithDB(ctx, instance) context.Context
gormz.WithGormDB(ctx, db) context.Context
gormz.DBFromContext(ctx) (*Instance, bool)
gormz.FromContext[T](ctx) *QuerySet[T]
gormz.MustFromContext[T](ctx) *QuerySet[T]
```

Advanced Package

Aggregates:

```
advanced.GroupBy[T](fields...) *AggregateQuery[T]
    .Count, .CountDistinct, .Sum, .Avg, .Min, .Max
    .Filter, .Where, .Having
    .OrderBy, .Limit, .Offset
    .All, .ScanInto, .CountGroups

advanced.Distinct[T](field) ([]any, error)
advanced.CountDistinctValues[T](field) (int64, error)
advanced.GroupConcat[T](field, sep) (string, error)
```

CTEs:

```
advanced.NewCTE[T](name, q) *CTE
advanced.NewRecursiveCTE[T](name, base) *CTE
advanced.With[T](ctes...) *CTEBuilder[T]
    .Query(q)
    .All
    .ScanInto
```

Window Functions:

```
advanced.RowNumber, Rank, DenseRank, Lag, Lead,
advanced.RunningSum, RunningCount, RunningAvg,
advanced.SumOver, CountOver, AvgOver,
advanced.NTile, FirstValue, LastValue, NthValue,
advanced.PercentRank, CumeDist
```

Subqueries:

```
advanced.SubFrom[T](q, field) *SubQuery
advanced.SubRaw(sql, args...) *SubQuery
advanced.In, NotIn, Exists, NotExists,
advanced.GtSub, LtSub, EqSub, GteSub, LteSub
advanced.NewCorrelated(sql, args...)
```

Unions:

```
advanced.Union[T] (queries...) *UnionQuery[T]
advanced.UnionAll[T] (queries...)
    .OrderBy, .Limit, .Offset
    .All
```

Locking:

```
advanced.WithLock[T] (q) *PessimisticQuery[T]
    .ForUpdate, .ForShare, .ForNoKeyUpdate, .ForKeyShare

advanced.WithOptimisticLock[T] (q) *OptimisticQuery[T]
    .Field, .UpdateIfVersion, .DeleteIfVersion, .Increment
```

Retry:

```
advanced.Retry(ctx, cfg, fn) error
advanced.RetryDB(ctx, fn) error
advanced.RetryWithBackoff(ctx, attempts, backoff, fn) error
advanced.ExponentialBackoff(initial, max) BackoffStrategy
advanced.LinearBackoff(step, max) BackoffStrategy
advanced.ConstantBackoff(delay) BackoffStrategy
```

Transactions:

```
advanced.Begin(ctx, cfg) (*Tx, error)
advanced.WithTransaction(ctx, cfg, fn) error
advanced.SimpleTransaction(ctx, fn) error

tx.Query[T] () *QuerySet[T]
tx.Commit() error
tx.Rollback() error
tx.RollbackIfActive()
tx.Nested(fn) error
```

Bulk:

```
advanced.BulkInsert[T] (ctx, items, cfg) error
advanced.BulkUpsert[T] (ctx, items, cfg) error
```

```
advanced.BulkUpdate[T](ctx, idCol, items) error
advanced.BulkDeleteByIDs[T](ctx, ids, size) (int64, error)
advanced.BulkDeleteWhere[T](ctx, where, args...) (int64, error)
advanced.BulkCount[T](ctx, where, args...) (int64, error)
```

Batch:

```
advanced.ProcessBatch[T](ctx, items, cfg, fn) *BatchResult
advanced.Stream[T](ctx, batchSize, fn) error
advanced.Parallel() *ParallelQuery
    .WithWorkers(n)
    .Add(fn)
    .Run(ctx)
```

Appendix B — FAQ

Q: Is gormz a replacement for GORM? A: No. gormz is a **library built on top of GORM**. It uses GORM under the hood but offers a cleaner, type-safe API. You can use both in the same project.

Q: Can I use GORM hooks? A: Yes! Since gormz uses GORM under the hood, all GORM hooks work:

```
func (u *User) BeforeCreate(tx *gorm.DB) error {
    u.Name = strings.TrimSpace(u.Name)
    return nil
}
```

Q: How do I handle errors? A: Use typed errors:

```
if gormz.IsNotFound(err) { /* ... */ }
if ne, ok := gormz.AsNotFound(err); ok { /* ... */ }
```

Q: Is QuerySet safe for concurrent use? A: Yes. QuerySet is **immutable** — every method returns a new copy.

Q: How do I use it with PostgreSQL? A:

```
import "gorm.io/driver/postgres"
db, _ := gorm.Open(postgres.Open(dsn), &gorm.Config{})
gormz.SetDB(db)
```

Q: What's the difference between Filter and Where? A:

- **Filter:** uses advanced lookups, validates field names (safe)
- **Where:** raw SQL, no validation (your responsibility)

Q: How do I do pagination? A:

```
page, _ := gormz.New[User]().Paginate(1, 20)
// page.Items, page.Total, page.Page, ...
```

Q: How do I use transactions? A:

```
err := gormz.Transaction(ctx, func(tx *gorm.DB) error {
    // ...
    return nil
})
```

Q: Is multi-tenancy supported? A: Yes — use a separate `Instance` for each tenant:

```
tenantDB := gormz.NewInstance(getTenantDB(tenantID))
users, _ := tenantDB.Query[User]().All()
```

Q: How do I migrate from GORM? A: See section 29. The migration is gradual.

Q: Is the library tested? A: Yes — **~90% coverage**.

Q: How do I contribute? A: See `CONTRIBUTING.md`.

Q: What's the license? A: MIT — free for commercial use.

Q: How do I report a security issue? A: See `SECURITY.md`. Do not

open a public issue.

Q: Does gormz support soft delete? A: Yes:

```
type User struct {
    ID          uint
    DeletedAt   gorm.DeletedAt `gorm:"index"`
}
```

Q: Does gormz support PostgreSQL? A: Yes — all GORM-supported databases.

Q: Can I use it with Fiber/Echo/Gin? A: Yes. gormz is framework-agnostic.

Q: How do I use gormz with Docker? A: Standard Docker practices — check `products-api` example.

Q: What's the recommended production setup? A: See section 28 (Performance Tuning).

End of gormz — Complete Reference

Version: 0.1.0 **License:** MIT **Author:** Sanad Team **Repository:** <https://github.com/light-tech-dev/gormz>

© 2025 Sanad Team

How to Save and Convert to PDF

Save the file

Copy all content above into `REFERENCE.md` in your project.

Convert to PDF

Option 1 — Pandoc (recommended):

```
pandoc REFERENCE.md -o gormz-reference.pdf \  
  --pdf-engine=xelatex \  
  --toc \  
  --toc-depth=3 \  
  --highlight-style=tango \  
  -V geometry:margin=1in \  
  -V fontsize=10pt \  
  -V mainfont="DejaVu Sans Mono" \  
  --number-sections
```

Option 2 — md-to-pdf (Node):

```
npm install -g md-to-pdf  
md-to-pdf REFERENCE.md
```